Trees and Ensembles — Commentary

Lesson 7 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about two and a half hours

Contents

Slide 1 — Lesson 7: Trees and Ensembles 5

Part I — One tree **5**
Slide 2 — Agenda 5
Slide 3 — Exercise 6, before we start 5
Slide 4 — 1,200 loan applicants, two figures each 6
Question — can a model score above the noise ceiling? 6
Slide 5 — Four risky rules, three risky regions 7
Question — why three regions if there are four rules? 7
Slide 6 — No straight line separates them 8
Question — why does logistic regression beat the baseline at all? 9
Question — which way does the line go? 9
Slide 7 — The whole algorithm, in one sentence 9
Question — is it "one feature, then the next"? 10
Question — when does the algorithm stop? 10
Slide 8 — The algorithm, drawn 11
Question — how was 0.82 chosen? Could it not have been 0.5? 12
Question — does 0.82 separate "most defaults" from "fewest"? 12
Question — where did the two groups of step 1 go? 12
Question — how should I read the left branch of step 2? 12
Question — why make a split that changes nothing? 12
Slide 9 — How pure is a group of examples? 13
Question — why is a fifty-fifty group the most impure? 13
Slide 10 — Gini impurity 13
Question — what are p_c and the classes? 14
Question — why square the shares? 14
Question — the 229 were split into 160 and 69: does that change 0.115? 14
Worked example — the Gini impurity of three groups of eight 14
Slide 11 — For two classes, one number 14
Slide 12 — The gain a split buys 15
Question — what does "worth taking if the children are purer on average" mean? . . 15
Slide 13 — The reduction in impurity 15
Question — how is the gain then used to go on? 16

Question — can a gain be negative? . . . 16
Worked example — the gain of one split of ten loans . . . 16
Slide 14 — Scanning for the best split . . . 16
Question — is this a summary of the whole algorithm? . . . 17
Question — why midpoints and not the values themselves? . . . 17
Slide 15 — Every threshold, scored . . . 17
Question — which panel is which? . . . 18
Question — why choose the lowest point and not the highest? . . . 18
Slide 16 — Worked example: the root split . . . 18
Question — where does 249 come from? . . . 19
Slide 17 — The root split, on the Gini curve . . . 20
Question — why is the gold square exactly under the parent? . . . 20
Slide 18 — The loop, in seven lines . . . 20
Question — what does `node` do? . . . 21
Question — is training "building the tree" and inference "walking it"? . . . 21
Slide 19 — Checked against scikit-learn . . . 22
Question — why compare predictions, and on the training data? . . . 22
Slide 20 — What greedy misses . . . 22
Question — is this why the islands are not found? . . . 22
Question — why can't the tree find the islands immediately? . . . 22
Slide 21 — Depth is the bias-variance dial . . . 23
Question — is that the whole picture? . . . 23
Question — why can an unconstrained tree reach 1.000 on noisy data? . . . 23
Slide 22 — Measured, at seven depths . . . 23
Question — how is the cross-validated accuracy computed, and what is the ±? . . . 24
Question — why is depth 4 worse than depth 2? . . . 24
Slide 23 — The curve bends downward . . . 25
Question — is the curve just the previous table? . . . 25
Slide 24 — The same rule, three depths . . . 26
Question — does it all add up? . . . 26
Question — why rectangles and not smooth curves? . . . 26
Slide 25 — Reading a tree, and the knob that keeps it small . . . 27
Question — is 90.8% the tree's accuracy? . . . 27
Question — how can nine leaves beat seventy-one? . . . 27
Worked example — which leaf a budget of leaves splits next . . . 28
Slide 26 — A tree that doesn't stay still . . . 28
Question — what does "without replacement" mean? . . . 28
Question — where do 26% and 94% come from? . . . 28
Question — why are the two thresholds different at all? . . . 29
Question — "strongest" and "weaker" splits? . . . 29
Slide 27 — Two trees, 140 different verdicts . . . 29
Question — are the 140 in the islands? . . . 30
Slide 28 — Notebook 1, live . . . 30
Slide 29 — Break . . . 30

Part II — Many trees that vote **30**
Slide 30 — Bagging: many trees, one vote . . . 30
Question — could the trees not "vote both ways" on a flipped label and so get it right? 31

Question — is bagging a trick to force disagreements? . . . 31
Question — is bagging to a deep tree what cross-validation is to logistic regression? 31
Worked example — five trees vote on three applicants . . . 32
Slide 31 — Bagging, drawn . . . 32
Question — why do the samples have different sizes? . . . 33
Question — does the bottom row show predictions on the training set? . . . 33
Slide 32 — How much data one bootstrap sample sees . . . 33
Question — 441 repeated? . . . 34
Worked example — one bootstrap sample of five rows . . . 34
Slide 33 — One bootstrap sample, counted against the theory . . . 34
Question — why are "never" and "once" almost equally likely? . . . 35
Question — what is on the axes? . . . 35
Slide 34 — Out-of-bag: each loan judged by the trees that never saw it . . . 36
Question — what if a loan is in every tree's sample? . . . 36
Slide 35 — Out-of-bag against cross-validation . . . 37
Question — is the area under the curve evaluated? . . . 37
Question — are the two methods the same thing? . . . 37
Slide 36 — Cross-validation, step by step: five groups of 240 . . . 37
Question — why divide by five and not four? . . . 38
Slide 37 — Why voting helps, and its floor . . . 38
Question — how can the 1,200 loans "say something wrong"? . . . 38
Slide 38 — The experiment: two sets of loans, two models, one applicant . . . 39
Question — changes with respect to what? . . . 39
Question — what are the "40 new sets"? . . . 40
Slide 39 — Another 1,200 loans: how many verdicts change? . . . 40
Question — does the flat part mean adding trees never helps? . . . 41
Slide 40 — Bagging against one tree, on 30 splits . . . 41
Question — is the spread about the same? . . . 42
Question — why 30 splits and not one? . . . 42
Slide 41 — Random forests: trees less alike . . . 42
Question — what is the intuition? . . . 43
Question — why deliberately deny a tree the best feature? . . . 43
Question — is more diversity always better? . . . 43
Slide 42 — Bagged trees against forest trees . . . 44
Question — so the forest diversifies the errors? . . . 44
Slide 43 — Random forests, measured . . . 45
Question — is the floor now 2.6%, and what are the boxes? . . . 45
Slide 44 — Feature importance on one tree . . . 45
Question — why does the root get more credit than the right split? . . . 46
Slide 45 — What a forest's importance says . . . 46
Question — is importance the same as correlation with default? . . . 46
Slide 46 — More columns, more noise . . . 47
Question — what does "more than half of the importance" mean? . . . 47
Question — why doesn't averaging 300 trees make the noise disappear? . . . 47
Slide 47 — Permutation importance: the procedure . . . 48
Question — why shuffle one column at a time? . . . 48
Slide 48 — Permutation importance: claimed against measured . . . 49
Question — how can a column have negative importance? . . . 49

Slide 49 — Notebook 2, live . . . 50

Part III — Trees in sequence: boosting **50**
Slide 50 — Boosting: one tree after another . . . 50
Question — is that the idea? . . . 50
Question — why regression trees inside a classifier? . . . 51
Slide 51 — Boosting, drawn: the first two steps . . . 51
Question — what does "a flat prediction, the mean of all y" mean? . . . 51
Question — why doesn't the first tree move the prediction all the way? . . . 51
Worked example — three boosting steps on four points . . . 52
Slide 52 — Building the ensemble, one tree at a time, for squared error . . . 52
Question — only formulas? . . . 52
Slide 53 — What each tree is fit to . . . 52
Question — why is the residual the negative gradient? . . . 53
Slide 54 — A step plus a sine wave, fit by 60 shallow trees . . . 53
Question — is the mean visible here? . . . 54
Question — how can shallow trees draw a curve? . . . 54
Slide 55 — Training error keeps falling; the true error does not . . . 54
Question — how can the true error rise while the training error falls? . . . 54
Slide 56 — Classification: the same idea, a different gradient . . . 55
Question — is this sigma the one of the variance formula? . . . 55
Worked example — where classification boosting starts . . . 55
Slide 57 — On the loan data . . . 55
Question — which data do the results refer to? . . . 56
Question — why is the default already competitive? . . . 56
Slide 58 — Boosting never stops fitting on its own . . . 56
Question — why can training reach 1.000 while cross-validation falls? . . . 56
Slide 59 — Training climbs to 1.000 and stays; the honest curve turns over first . . . 57
Question — how do I choose when to stop, if the truth is unknown? . . . 57
Slide 60 — Fixing 120 trees, varying only the learning rate . . . 58
Slide 61 — Notebook 3, live . . . 58

Part IV — Closing **58**
Slide 62 — Every method, on the same five folds . . . 58
Question — is the forest definitely better than bagging? . . . 59
Slide 63 — Three findings . . . 59
Question — why is a nine-leaf tree so close to the ensembles? . . . 59
Slide 64 — How to choose, in practice . . . 60
Question — is the random forest always the best choice for tabular data? . . . 60
Slide 65 — 54% of the importance on noise . . . 60
Question — why does noise still get 28% with every column on offer? . . . 61
Worked example — how often the menu holds no real column . . . 61
Slide 66 — What to take away . . . 61
Slide 67 — Homework . . . 62

What this document is. A companion to the slides of lesson 7: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It

is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/`. Where a section mentions "the notes", it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide. "The rule that generated the data" always means the rule of slide 5, which `loan_data.py` applies before it flips 7% of the labels.

Worked examples sit under eight slides (10, 13, 25, 30, 32, 51, 56, 65): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Slide 1 — Lesson 7: Trees and Ensembles

The title slide. The lesson has one dataset, 1,200 loan applicants, and one question asked in three ways: what one decision tree can do, what many trees voting can do, and what many trees built one after another can do. The thread that ties the three together is **instability**: a single tree is easy to read and easy to upset, and everything after the break is about turning that weakness into strength.

Part I — One tree

Slide 2 — Agenda

Lessons 3 and 4 fitted a line; lesson 6 offered three ways of drawing a boundary a line cannot draw (k-nearest neighbours, Naive Bayes, the support vector machine). Lesson 7 adds a fourth kind of model, built on a different basic step: **compare one feature with a threshold**. k-nearest neighbours measured distances, the support vector machine computed kernels; a tree asks yes-or-no questions.

Most of the three hours, though, go to a different question: not "does a tree work" but "what happens when you build more than one". The agenda's second line says why: a single tree is readable, and on its own it is **unstable**: train it on another sample of the same applicants and it gives other verdicts.

Slide 3 — Exercise 6, before we start

The lesson opens by discussing exercise 6, where you compared the three families of lesson 6 and had to defend one. What is judged is the **defence**, not the score. The common gap is a good cross-validated accuracy quoted with no check of whether the model's assumption held on the data: for Naive Bayes, for instance, whether the features really were independent within each class.

The slide's last line carries the obligation forward. Random forests come with an accuracy and with a ranking of the columns by importance; by the end of the lesson you will have measured how far that ranking can mislead when most of the columns carry no information.

Slide 4 — 1,200 loan applicants, two figures each

The dataset of the whole lesson. Each applicant has two figures: **income**, in thousands of euros a year, and **debt-to-income ratio**, between 0 and 1. The label says whether the loan later defaulted.

The lender's logic is a **checklist**, not a smooth function: an income floor (below it, risky), a debt ceiling (above it, risky), and two "stressed" combinations where income and debt are each unremarkable on their own but risky together. Then **7% of the labels are flipped at random**, to stand for everything the two figures cannot know: a job lost, an inheritance.

Two numbers frame every result that follows:

- **The majority baseline, 0.613.** 38.7% of the applicants defaulted, so a model that always answers "repaid" is right on $1 - 0.387 = 0.613$ of them. Anything below that has learned nothing.
- **The noise ceiling, about 0.93.** A flipped label is a coin toss no model can foresee, so even the rule that generated the data is right, on average, on $1 - 0.07 = 0.93$ of new applicants. In this particular sample 85 of the 1,200 labels were flipped, so the rule itself scores $1{,}115/1{,}200 = 0.929$.

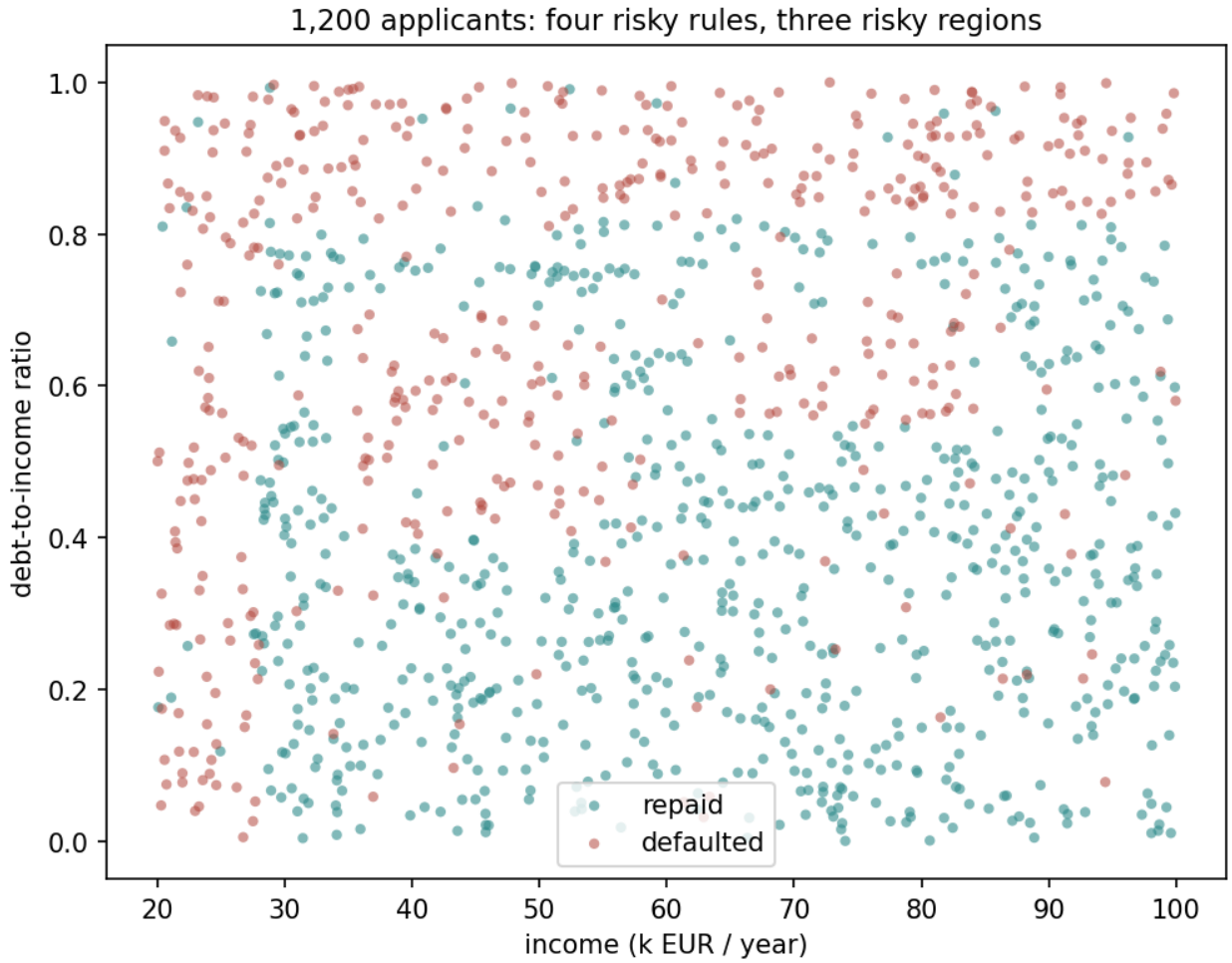
Question — can a model score above the noise ceiling?

> *If 7% of the labels are noise, is 0.93 a hard limit? What should I think if a model scores 0.94?*

On the data it was **trained** on, yes it can, and easily: a deep tree memorises the flipped labels and reaches 1.000 (slide 23). That is not a bug, it is memorising.

On **new** data, 0.93 is an average, not a wall. The number of flipped labels in a test set varies from one set to another, so even the rule that generated the data scores above 0.93 on about half of all possible sets of 1,200 new applicants (probability 0.484). So a held-out score a little above the ceiling can be luck. A score far above it, or above it on every split, means something is wrong: usually information about the label leaking into the inputs, as in lesson 1.

Slide 5 — Four risky rules, three risky regions



The 1,200 applicants, rust for defaulted and teal for repaid. The risky area is an L along the left and top edges, plus two rectangles inside.

The scatter shows the shape the checklist makes. Four rules make an applicant risky: income below the floor, **or** debt above the ceiling, **or** income and debt inside one of two stressed rectangles. The low-income strip runs up the left edge and the high-debt strip along the top; they meet in the top-left corner, so together they are **one L-shaped region**. The two rectangles, the "islands", stand apart in the interior. Four rules, three separate regions.

A predictable mistake. Reading the floor and the ceiling as "low income **and** high debt", which is one corner of the plane, instead of "low income **or** high debt", which is the whole L. The reading is reasonable, because the two islands really are an "and" of two ranges. The floor and the ceiling are not: each applies on its own.

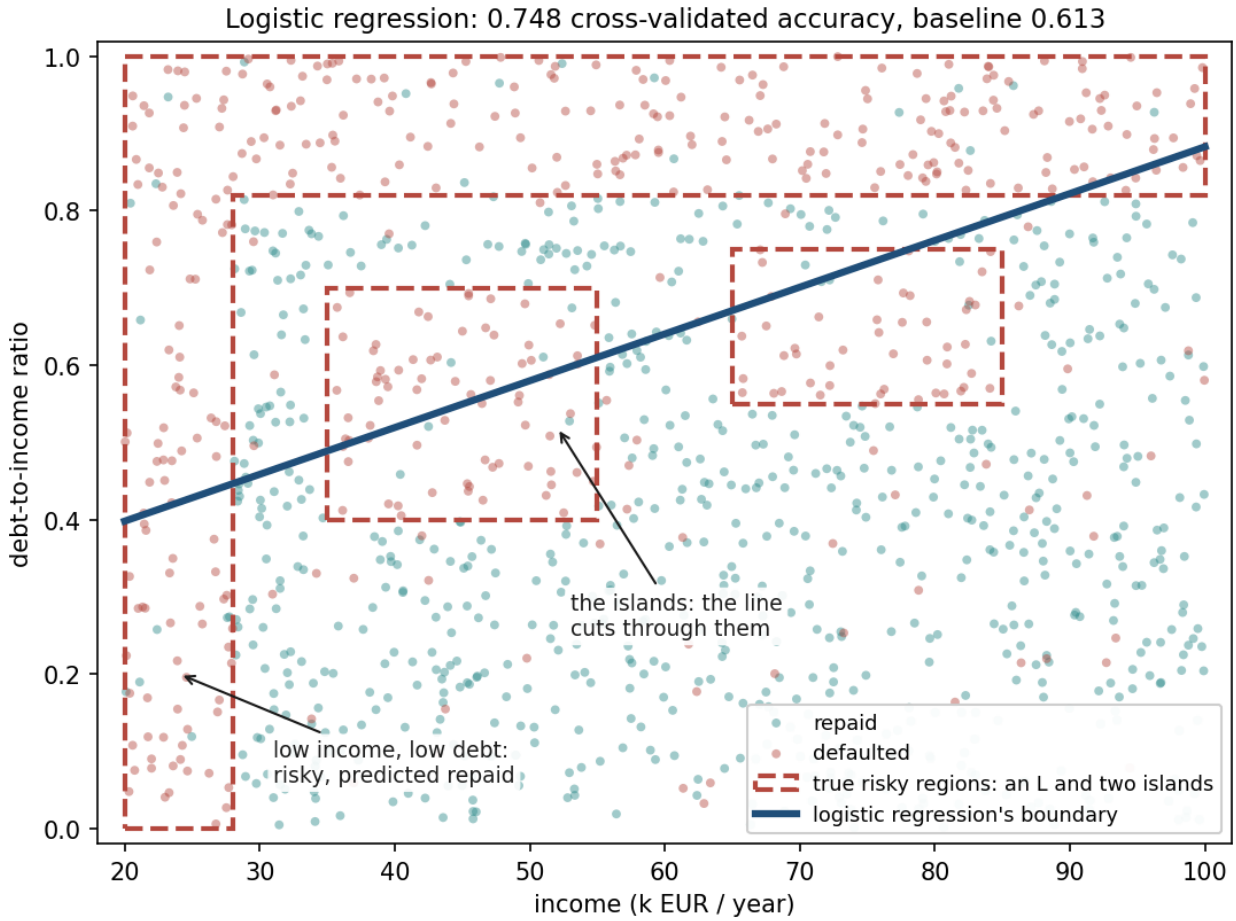
Question — why three regions if there are four rules?

> *The rule has four conditions. Why does the slide say three regions?*

Because two of the conditions touch. Every applicant below the income floor is risky, whatever the debt; every applicant above the debt ceiling is risky, whatever the income. The two strips therefore

overlap in the top-left corner and form one connected L. Each island is a separate rectangle. One L plus two islands is three regions.

Slide 6 — No straight line separates them



The line logistic regression fits on all 1,200 applicants, and the true risky regions outlined. Above the line the model predicts default.

Logistic regression, lesson 4's model, draws one straight line. Fitted on the 1,200 applicants it runs from a debt ratio of 0.40 at an income of 20 thousand euros to 0.88 at 100 thousand: above the line, predicted default. It scores **0.748 ± 0.030** in five-fold cross-validation, well above the baseline of 0.613.

So the line has learned something real: most of the high-debt strip lies above it, and there the risk rises steadily as one feature moves one way, which a straight boundary handles well. But look where it fails:

- **Bottom left.** Income below the floor with low debt is risky, yet it lies below the line; of the 87 applicants there, the model calls 50 "repaid".
- **Far right.** At high income the line climbs above the ceiling, so debt between 0.82 and 0.88 is called "repaid".

- **The islands.** The line cuts straight through both, calling part of each healthy.

No single line can follow an L and two islands. What can is a threshold rule applied more than once, which is what a tree is.

Question — why does logistic regression beat the baseline at all?

> *If no straight line separates the classes, why is it at 0.748 and not at 0.613, like the linear models on lesson 6's pumps?*

Because here a line captures part of the pattern. Low income and high debt both raise the risk, so a line that leans the right way puts most of the high-debt strip on the correct side. On lesson 6's pumps the faulty class surrounded the healthy one on every side, and any line left as many errors on one side as it fixed on the other. Here the line fails only on the parts it cannot bend around: the corner of the L and the islands.

Question — which way does the line go?

> *The line that separates "low income, high debt" from the rest: does it start low on the left and rise to the right?*

Yes, that is its direction: debt 0.40 at an income of 20 thousand euros, 0.88 at 100 thousand. It is almost a horizontal threshold on debt, tilted a little. But the risky region is "low income **or** high debt", an L, and a tilted line can only cut across the L's corner: it misses the bottom of the low-income strip and the top-right part of the high-debt strip.

Slide 7 — The whole algorithm, in one sentence

A decision tree is grown by one procedure, repeated:

1. **Try every feature and every threshold**, and keep the split that leaves the two groups as **pure** as possible (how purity is measured comes on slides 9 to 13).
2. **Apply it**, and repeat the search on each of the two groups, independently.
3. **Stop** a branch when it reaches a depth limit, when its group is pure, or when it has too few examples to split.

There are no coefficients and no gradient: the thresholds are found by **searching** among candidates, not by descending a slope. The tree still learns from the data, which feature to ask about and where to put each threshold, but it learns them by trying and keeping the best.

The word to hold onto is **greedy**: at each step the tree takes whichever split helps most right now, with no way of looking two steps ahead. Slide 20 shows what that costs.

A predictable mistake. "One feature per level, applied to all the applicants, then the next feature." It is natural, because a drawn tree looks like a fixed sequence of questions, and the first question really is asked of everyone. It is wrong twice over: after the first split each group is searched on its own, so two groups can get different questions; and a feature can come back, as debt does on slide 8.

Question — is it "one feature, then the next"?

> *You start with a feature, choose a threshold, and divide all the samples into two groups according to whether they are above or below it. Then you change feature, choose another threshold and divide the samples again. Is that right?*

Half right. The split itself is right: a threshold, two groups, at-or-below and above. Three corrections:

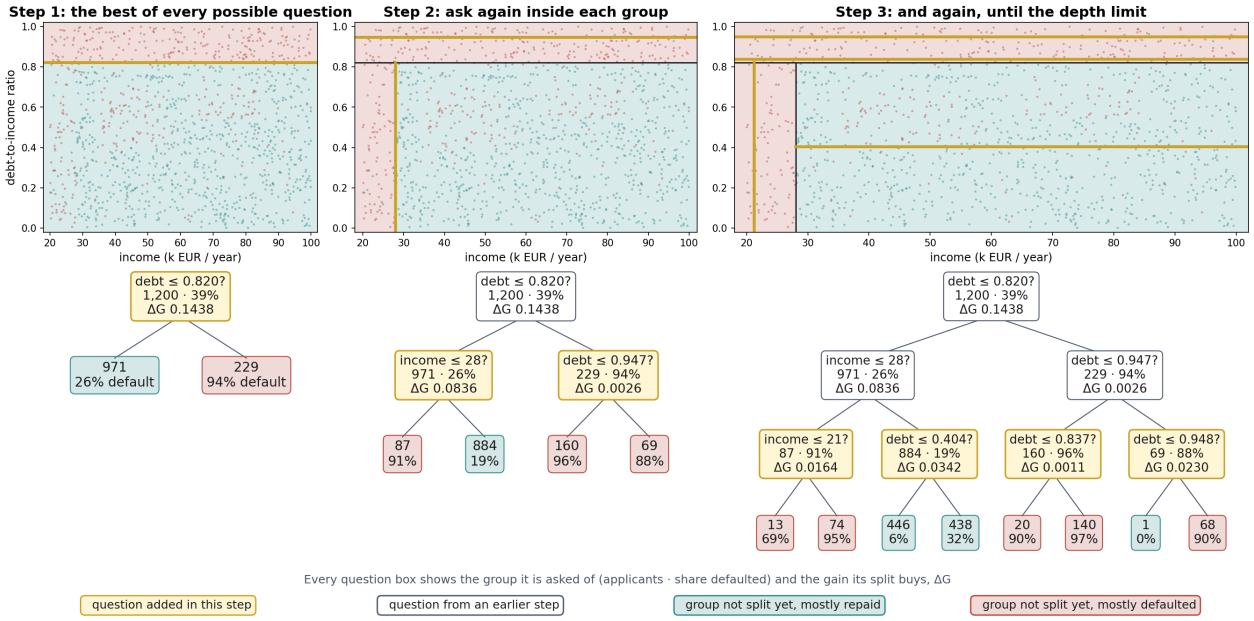
1. **Nobody chooses the feature first.** At every node the tree tries every feature and every threshold and keeps the pair that leaves the purest groups. At the root it chose debt ratio at 0.82 over every income threshold.
2. **The next step does not re-split all the samples.** Each of the two groups is searched on its own, and the two may get different questions: on slide 8, income on the left, debt on the right.
3. **"Change feature" is not a rule.** The same feature can come back: debt is asked about again on the right branch.

Question — when does the algorithm stop?

> *Impurity is 0 when a group is pure. When a group becomes pure there is no need to go on splitting. The algorithm stops when all groups are pure, or when it reaches a maximum depth. Correct?*

Yes, with one refinement: the stopping rules act **branch by branch**. A pure group stops its own branch while the others carry on, and the algorithm ends when every branch has stopped. The rules are the three on the slide (depth limit, pure group, too few examples, by default fewer than 2), plus one more case: no split helps, for example when the remaining examples have identical features and different labels. Without a depth limit every branch runs until its group is pure, which is why an unconstrained tree reaches a training accuracy of 1.000.

Slide 8 — The algorithm, drawn



The procedure run three times on the loan data. Top: the cuts in the plane, the new ones in gold. Bottom: the tree that asks the same questions. Every box shows the group it holds (applicants · share defaulted) and, for a question, the gain its split buys, ΔG.

Read each column top and bottom together: every gold line in the plane is a gold box in the tree.

- **Step 1.** Every possible single question was tried, 2,234 candidates in all, and "debt ratio at or below 0.82" leaves the purest groups: 971 applicants at 26% default against 229 at 94%.
- **Step 2.** Each of the two groups is searched again, on its own. The 971 get "income 28?" and split into 87 + 884; the 229 get "debt 0.947?" and split into 160 + 69. The two questions differ because the two searches are independent.
- **Step 3.** One more round. Neither interior island has appeared yet: three rounds went to the easy strips.

Two things in the figure are worth stopping on, because they show the algorithm's character.

A split can change no prediction. On the right of step 2, the 229 at 94% become 160 at 96% and 69 at 88%: both groups are still mostly defaulted, so both still predict "default". The tree makes the split anyway, because it judges **impurity**, not decisions, and any gain above zero is taken. Its gain is 0.0026, against 0.0836 for the income question on the left: about thirty times smaller.

The third level memorises one label. Its four splits change exactly one prediction in 1,200: training accuracy goes from 0.8400 at depth 2 to 0.8408 at depth 3. And that one applicant is a flipped label: income 23.22, below the floor, and debt 0.9475, above the ceiling, so both rules say default, yet the label says repaid. The tree spent a split on noise. One of those splits sets aside a single applicant, the cut at 0.948 just above the 0.947 one; in the plane the two gold lines lie on top of each other.

Question — how was 0.82 chosen? Could it not have been 0.5?

> *The tree starts with debt 0.820. Could I not choose a different threshold, 0.5 for example? Or did the algorithm try them all?*

It tried them all. The candidates are the midpoints between neighbouring values in the data: 1,138 for debt ratio and 1,096 for income, 2,234 at the root. Each is scored by how pure the two groups it makes are (slides 9 to 15). Debt at 0.5 splits 627 applicants at 18% default from 573 at 61%: both groups stay mixed. Debt at 0.82 leaves 971 at 26% and 229 at 94%, the purest pair of all 2,234. Slide 15 draws the whole search.

Question — does 0.82 separate "most defaults" from "fewest"?

> *So 0.820 is the threshold that best separates the samples into two classes, one with the most defaults and one with the fewest?*

Yes on the search, with two refinements. The split makes two **groups**, not two classes: each group still holds both classes, 26% defaulted on the left and 94% on the right. And the criterion is **proportions weighted by size**, not numbers of defaults: slide 15 shows a threshold, debt at 0.9, that makes a right group almost as pure but loses because it leaves the large left group more mixed.

Question — where did the two groups of step 1 go?

> *In step 2 I expected to see the two groups of step 1 again; instead I see two gold boxes under the root.*

They are still there. Each group's box has been replaced by the question asked of it: the 971 became "income 28?", the 229 became "debt 0.947?", and each box still shows its group (971 · 26%, 229 · 94%). Their children add up: 87 + 884 = 971 and 160 + 69 = 229. A group that is split becomes a question; its applicants do not go anywhere.

Question — how should I read the left branch of step 2?

> *On the left branch, 971 applicants at 26%: the algorithm tried every threshold on both features and found that splitting on debt 28 gave the purest groups, 87 at 91% on the left and 884 on the right of whom only 19 defaulted. Right?*

Right on the mechanism: a search inside the 971 only, on both features, every threshold made from those 971 values. Two slips of reading. The question is on **income** at 28 (thousand euros), not debt: debt is a ratio between 0 and 1, and 28 is the income floor found again. And 19 is **19%**, not 19 people: 170 of the 884 defaulted. The counts close: 87 with 79 defaults plus 884 with 170 is 249 defaults, the 26% of 971.

Question — why make a split that changes nothing?

> *Why does the tree split groups whose two halves both predict the same class?*

Because the splitting criterion is the reduction of Gini impurity, not an improvement in accuracy. Two groups can have different shares of defaulters while both stay majority-default: 96% and 88% are purer, on average, than 94% all together, so the impurity goes down a little and the tree takes the gain. Whether a split was worth making is a question the tree cannot ask; slide 25 shows the knob that lets you ask it.

Slide 9 — How pure is a group of examples?

Before any formula, the idea. A group whose members all share a label needs no further questions. A group half one class and half the other is the worst case: guessing its label is a coin flip. **Impurity** is a number that captures this, 0 for a pure group and highest for an even mix, and "the purest groups" on slide 7 means "the groups with the lowest impurity".

A predictable mistake. "This split is impure." Impurity belongs to a **group**, not to a split. A split is judged by the impurity of the two groups it makes, weighted by their sizes (slides 12 and 13).

Question — why is a fifty-fifty group the most impure?

Why should a group split evenly between the classes count as the worst?

Because neither class dominates. If you drew two members of the group at random, a fifty-fifty group gives the highest chance that they have different labels; a pure group gives zero. That chance is exactly what the Gini impurity of the next slide measures.

Slide 10 — Gini impurity

The standard measure of impurity for classification:

$$G = 1 - \sum_{c=1}^{C} p_c^2$$

- The **classes** are the labels a group can hold: here two, defaulted and repaid, so $C = 2$.
- p_c is the **share** of the group in class c; the shares add up to 1.
- $\sum_c p_c^2$ is the chance that two members drawn at random (with replacement) have the **same** label; one minus it is the chance they **differ**.

The slide works it on a group seen on slide 8: the 229 applicants of step 1 on the right, before they are split again. 215 of them defaulted, so $p = 215/229 = 0.939$ for defaulted and $14/229 = 0.061$ for repaid: $G = 1 - (0.939^2 + 0.061^2) = 1 - 0.885 = 0.115$. Nearly pure. The single repaid applicant set aside in step 3 is a pure group: $1 - (0^2 + 1^2) = 0$.

The formula never asks which class is common, only how mixed the group is: a group pure in "repaid" scores 0 exactly as one pure in "default" does.

A predictable mistake. "Gini never exceeds 0.5." Reasonable, since every group in this lesson has two classes, and with two classes 0.5 is the maximum. With C equally frequent classes, $G = 1 - C \times (1/C)^2 = 1 - 1/C$, so three classes reach 0.667.

A note of history. The name is Italian. Corrado Gini (1884–1965) was a statistician and the first president, in 1926, of the Istituto Centrale di Statistica, the body that became ISTAT. $1 - \sum p_c^2$ is his index of heterogeneity, "mutabilità", from 1912; Breiman, Friedman, Olshen and Stone made it the splitting criterion of their Classification and Regression Trees (CART) in 1984. It is not the Gini coefficient of income inequality, his better-known index: same man, a different quantity.

Question — what are p_c and the classes?

The formula as written does not say what p_c is, or who the classes are.

The classes are the possible labels: here "defaulted" and "repaid". p_c is the share of the group with label c. For the 229 applicants on the right of slide 8, p is 0.939 for defaulted and 0.061 for repaid, and the two add up to 1.

Question — why square the shares?

Why does the formula square each class's share?

Because p_c^2 is the probability that two members drawn independently from the group both belong to class c. Summed over the classes it is the probability that the two share a label, whatever the label; subtracted from one, the probability that they disagree. A pure group never disagrees with itself, so its impurity is 0.

Question — the 229 were split into 160 and 69: does that change 0.115?

The group of 229 was split into 160 at 96% and 69 at 88%. Does the 0.115 still hold?

It holds for the 229 **before** the split. After it there are two groups: 160 at 0.9625 default (impurity 0.072) and 69 at 0.884 (impurity 0.205). Weighted by size, $160/229 \times 0.072 + 69/229 \times 0.205 = 0.112$. The gain of that split is $0.115 - 0.112$, about 0.003: the 0.0026 printed on slide 8.

Worked example — the Gini impurity of three groups of eight

group	defaulted	repaid	p = defaulted ÷ size	$G = 1-(p^2+(1-p)^2)$
A	8	0	$8 \div 8 = 1.00$	$1 - (1.00^2 + 0.00^2) = 0.000$
B	6	2	$6 \div 8 = 0.75$	$1 - (0.75^2 + 0.25^2) = 0.375$
C	4	4	$4 \div 8 = 0.50$	$1 - (0.50^2 + 0.50^2) = 0.500$

A pure group scores 0, an even one 0.5, the two-class maximum. Each line also equals $2p(1-p)$, slide 11's shortcut. With three classes the ceiling moves: a group of nine with three of each class scores $1 - 3 \times (1/3)^2 = 0.667$.

Slide 11 — For two classes, one number

With two classes the shares are p and $1-p$, and the formula collapses:

$$1 - (p^2 + (1-p)^2) = 1 - p^2 - (1 - 2p + p^2) = 2p - 2p^2 = 2p(1-p)$$

It is **0** when the group is pure ($p = 0$ or $p = 1$) and **0.5**, its maximum, at $p = 0.5$. On the full 1,200 applicants, $p = 0.387$ defaulted, so $G = 2 \times 0.387 \times 0.613 = 0.474$: the impurity every candidate root split is measured against.

Slide 12 — The gain a split buys

A split divides a parent group P into a left child L and a right child R. Write m for the number of examples in each: $m_P = m_L + m_R$, and at the root $1{,}200 = 971 + 229$.

The split is worth taking if the two children's impurities, **averaged**, are lower than the parent's. At the root:

- left: 249 of the 971 defaulted, $p = 0.256$, $G = 2 \times 0.256 \times 0.744 = 0.381$;
- right: 215 of the 229, $p = 0.939$, $G = 2 \times 0.939 \times 0.061 = 0.115$;
- weights: $971/1{,}200 = 0.809$ and $229/1{,}200 = 0.191$;
- weighted average: $0.809 \times 0.381 + 0.191 \times 0.115 = 0.308 + 0.022 = 0.330$, lower than the parent's 0.474.

The average is **weighted by size** for a reason the last bullet makes concrete. Cut a single example off a mixed group of 1,200: the one-example child is pure, impurity 0, and the other is as mixed as the parent was. A plain average of the two would halve the impurity and make the split look excellent. Weighted, the pure child counts for 1/1,200 of the average, and the gain is close to zero, as it should be: nothing useful was learned.

Question — what does "worth taking if the children are purer on average" mean?

What exactly is compared with what?

The parent's impurity, as it is, against the children's impurities averaged with weights equal to each child's share of the parent. At the root, 0.474 against 0.330: lower, so the split helped. The parent's 0.474 is not weighted; it is the single number the weighted average is compared with.

Slide 13 — The reduction in impurity

The previous slide's comparison, as a formula:

$$\Delta G = G_P - \left(\frac{m_L}{m_P} G_L + \frac{m_R}{m_P} G_R \right)$$

The parent's impurity minus the children's impurities, each weighted by its share of the parent. ΔG is the **gain**; the tree keeps whichever candidate split makes it largest. At the root, $0.474 - 0.330 = 0.144$.

A predictable mistake. Subtracting the weighted average from 1 instead of from the parent's impurity: $1 - 0.330 = 0.670$ instead of $0.474 - 0.330 = 0.144$. It is natural, because the Gini formula three slides back begins with "1 −". The check that catches it: a split cannot remove more impurity than the parent had, so a gain never exceeds G_P, and with two classes never exceeds 0.5.

Question — how is the gain then used to go on?

I don't see how the gain is then used to carry the algorithm forward.

ΔG does two jobs, at one node at a time. It **ranks** that node's candidates: the largest gain wins. And it **decides whether to split at all**: if no candidate gains anything, the node becomes a leaf. Then each child repeats the whole procedure on its own examples, with its own impurity as the parent's and weights relative to its own size. Walk slide 8's tree: at the root, 2,234 candidates, the best gains 0.1438, so split; the left group of 971 searches again, best gain 0.0836, split; the right group of 229, best gain 0.0026, small but above zero, split; at the depth limit, stop. In this loop gains are never added up or compared between nodes; slide 25 shows the one setting that compares them.

Question — can a gain be negative?

Could a split make things worse, a negative ΔG?

Not for a split that divides a group in two, as these do. The parent's share of defaulters is exactly the size-weighted average of the children's shares (defaulters are moved, not created), and because the Gini curve bulges upwards, the weighted average of the children's impurities can never lie above the parent's. A candidate can gain zero, if both children keep the parent's exact share; the algorithm then makes no split, because it takes only a gain above zero. Slide 17 draws why.

Worked example — the gain of one split of ten loans

Ten loans, 4 defaulted. A threshold sends 4 loans left (3 defaulted) and 6 right (1 defaulted).

group	loans	defaulted	$G = 2p(1-p)$	weight = loans ÷ 10
parent	10	4	$2 \times 0.40 \times 0.60 = 0.480$	
left	4	3	$2 \times 0.75 \times 0.25 = 0.375$	0.4
right	6	1	$2 \times 0.167 \times 0.833 = 0.278$	0.6

Weighted average of the children: $0.4 \times 0.375 + 0.6 \times 0.278 = 0.317$. Gain: $\Delta G = 0.480 - 0.317 = \mathbf{0.163}$. The slip of subtracting from 1 would give 0.683, larger than the parent's whole impurity, 0.480: impossible, which is how to catch it.

Slide 14 — Scanning for the best split

How the candidate thresholds are found. Sort a feature's values; every **midpoint** between two neighbouring values is a candidate. Score every candidate, on every feature; keep the best pair; repeat inside each child.

A threshold is a real number, so are there not infinitely many to try? There are, but they make only **finitely many different splits**. At the root, two neighbouring debt ratios in the data are 0.8198 and 0.8205, with no applicant between them; any threshold from 0.8198 up to just below 0.8205 sends the same 971 applicants left and the same 229 right. So one threshold per gap is enough, and the convention is the midpoint, $(0.8198 + 0.8205)/2 = 0.82015$, printed 0.820: halfway, as far

as possible from both values seen in training. Debt ratio has 1,139 distinct values, so 1,138 gaps; income 1,097, so 1,096. The search is exact.

A predictable mistake. "The search must be approximate, because thresholds are continuous." The data are finite, so the splits are too.

The last bullet is the important one: ΔG is judged **one split at a time**, never for two splits together. Slide 20 shows what that misses.

Question — is this a summary of the whole algorithm?

> *Sort the values, take the midpoints, loop over all thresholds and all features, find the pair with the largest reduction of impurity, then repeat for each child. And "greedy" means the reduction is judged at the current level only, without looking further down the branch for a better solution. Correct?*

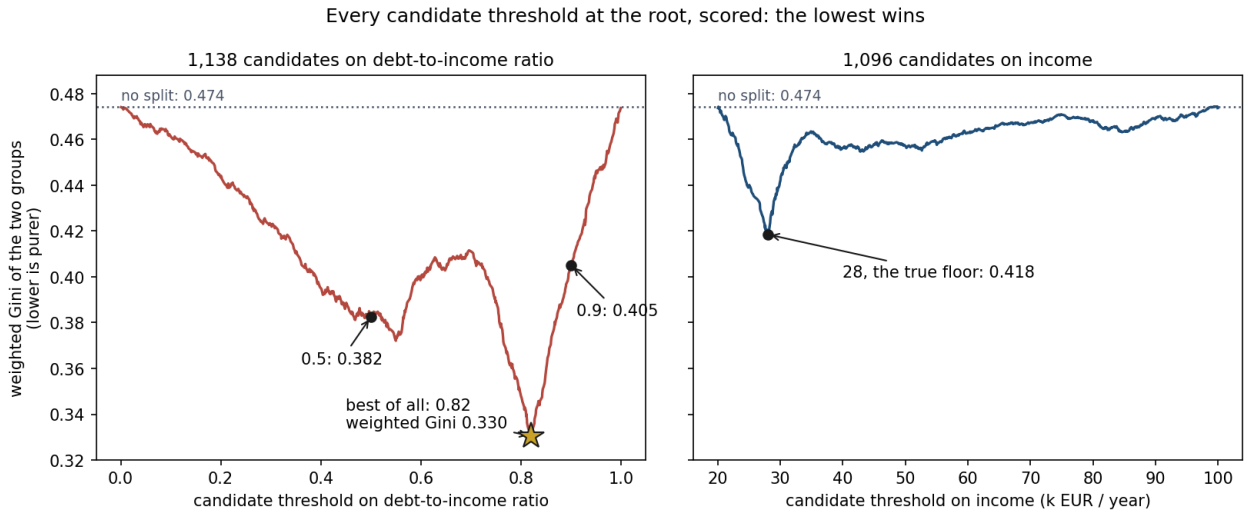
Correct, with two details: the midpoints are between neighbouring **distinct** values, and each feature is sorted separately. Greedy has two halves. Each split is chosen by its own gain, never by what it would allow one or two levels down; and once made it is never revisited. The alternative would be expensive: judging pairs of splits together would mean about $2{,}234^2$, some five million, pairs at the root alone.

Question — why midpoints and not the values themselves?

> *Why not use the observed values as thresholds?*

Any threshold between the same two neighbours makes the same split, so the choice inside the gap does not change the training result. It matters for new applicants, who can fall inside the gap: the midpoint keeps the boundary as far as possible from both training values, rather than right on top of one of them.

Slide 15 — Every threshold, scored



Each point is one candidate threshold at the root, its height the weighted Gini impurity of the two groups it makes. Left, the 1,138 on debt ratio; right, the 1,096 on income. The dotted line, 0.474, is no split at all.

The previous slide's search, drawn. Lower means purer. The search keeps the lowest point on either curve: debt at 0.82, 0.330. The gain is the drop from the dotted line to that point, $0.474 - 0.330 = 0.144$.

The alternatives make the criterion concrete:

- **Debt at 0.5** splits 627 applicants at 18% default from 573 at 61%: both groups stay mixed, 0.382.
- **Income's best**, close to the true floor of 28, scores only 0.418: the low-income strip holds 111 applicants and leaves the other 1,089 mixed.

A predictable mistake. "The best split puts the most defaults on one side." A natural reading of "purest", but the score counts proportions weighted by size, not numbers of defaults. Debt at 0.9 makes a right-hand group almost as pure as 0.82 does, 129 applicants at 92% default against 229 at 94%, yet scores 0.405 against 0.330, because it leaves the large left group more mixed: 1,071 at 32% against 971 at 26%.

A second one. "The tree has recovered the lender's threshold, 0.82." The tree does not know the rule that generated the data. It takes the midpoint of two neighbouring debt values in the data, 0.8198 and 0.8205, and that midpoint happens to round to the same 0.82. The agreement is evidence that the search works, not knowledge of the rule.

Question — which panel is which?

> *The panel on the right, debt at 0.82 with 0.330, is lower than the one on the left, income at 28 with 0.418, so debt is chosen. Right?*

Right on the conclusion, with the panels swapped: debt is the **left** panel, income the right. In gains: $0.474 - 0.330 = 0.144$ for debt, $0.474 - 0.418 = 0.056$ for income. 28 is income's own best threshold, so the comparison is best against best.

Question — why choose the lowest point and not the highest?

> *The tree wants the biggest gain. Why does it pick the lowest point of the curve?*

Because the curve plots the impurity **left after** the split, not the gain. The parent's impurity is fixed at 0.474 for every candidate, so the lowest remaining impurity is the same thing as the largest drop from 0.474.

Slide 16 — Worked example: the root split

The root split, every step from a count:

Group	Applicants	Defaulted	p = defaulted ÷ applicants	$G = 2p(1-p)$
Parent	1,200	464	$464 \div 1{,}200 = 0.387$	$2 \times 0.387 \times 0.613 = 0.474$
Left, debt 0.82	971	249	$249 \div 971 = 0.256$	$2 \times 0.256 \times 0.744 = 0.381$
Right, debt > 0.82	229	215	$215 \div 229 = 0.939$	$2 \times 0.939 \times 0.061 = 0.115$

The counts close: $249 + 215 = 464$, every default. The weights are each child's share of the parent, $971/1{,}200 = 0.809$ and $229/1{,}200 = 0.191$; the weighted average is $0.809 \times 0.381 + 0.191 \times 0.115 = 0.330$, and the gain $0.474 - 0.330 = 0.144$.

Two warnings for anyone redoing it by hand. With four decimals the impurities are 0.3814 and 0.1148, the weighted average 0.3305 and the gain $0.4743 - 0.3305 = 0.1438$; with the rounded 0.381 and 0.115, the average comes out at 0.3302. Rounding, not an error. And the search found the debt ceiling with no knowledge of how the data were built: nothing in the algorithm knows the rule that generated the data, so the agreement is a real check that the code does what it should.

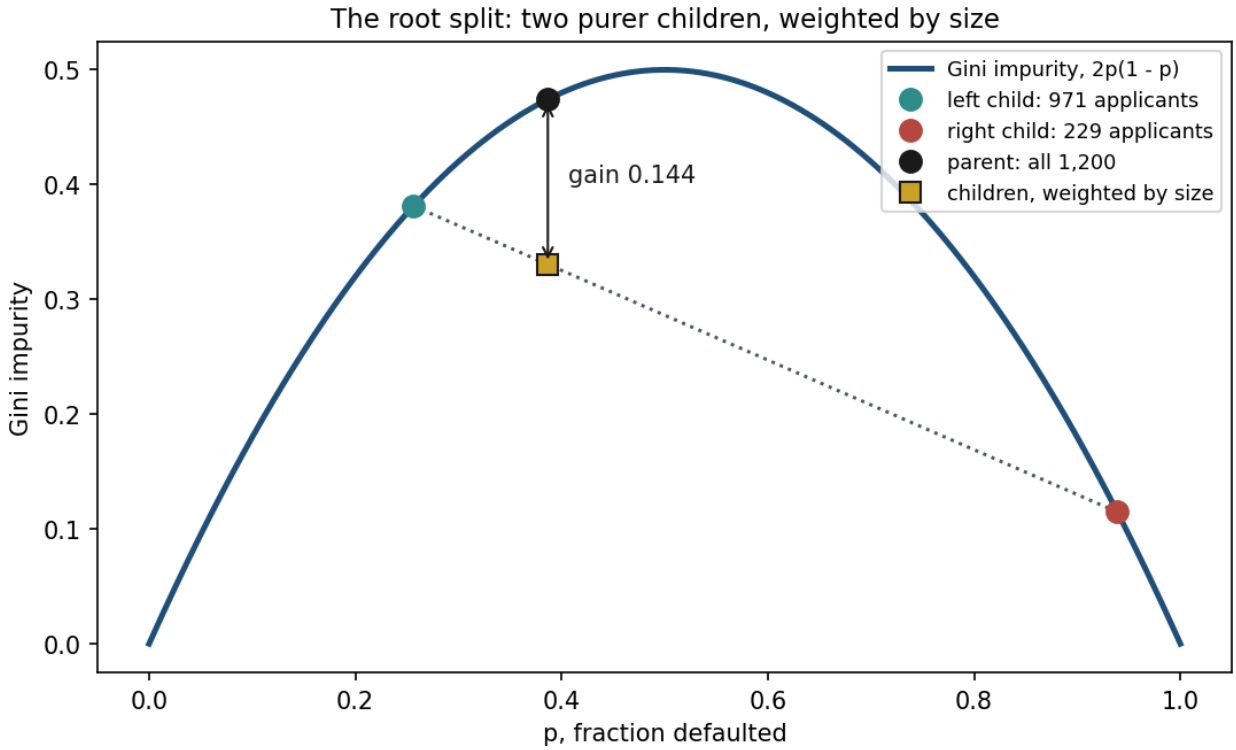
A predictable mistake. Rebuilding a count from a rounded percentage. Slide 8 prints the left group as 26% default, and $0.26 \times 971 = 252$, not 249: the true share is $249/971 = 25.6\%$. Counts come first; a share rounded to two digits loses up to half a percent of the group.

Question — where does 249 come from?

> *I computed 26% of 971 and got 252, not 249.*

249 is counted in the data: the applicants with debt at or below 0.82 whose label is "default". The share comes from the count, not the other way round: $249/971 = 0.2564$, which slide 8 rounds to 26%. Multiplying back the rounded 26% multiplies the rounding error by 971. The same happens on the right: $0.94 \times 229 = 215.3$, true count 215.

Slide 17 — The root split, on the Gini curve



The curve $2p(1-p)$. The black dot is all 1,200 applicants; teal and rust are the two children; the gold square is their size-weighted average, on the dotted line between them.

The previous slide's arithmetic as a picture. The parent sits at $p = 0.387$, impurity 0.474. The split sends 971 applicants to $p = 0.256$ and 229 to $p = 0.939$. The gold square, the two children averaged by size, sits at 0.3305, straight under the parent. The vertical gap, 0.144, is the gain.

Question — why is the gold square exactly under the parent?

Why does the square sit at the same horizontal position as the parent?

Because a split moves defaulters from one group to another; it does not create or remove any. The children's shares, averaged with the same weights, give the parent's share back, and with counts it is exact: $971/1{,}200 \times 249/971 + 229/1{,}200 \times 215/229 = 249/1{,}200 + 215/1{,}200 = 464/1{,}200 = 0.387$. So parent and square share one horizontal position, and the square lies on the straight line between the two children. Because the curve bulges above every such line, any split that separates the classes at all lowers the weighted impurity: that is also why a gain cannot be negative.

Slide 18 — The loop, in seven lines

Slides 7 to 17 in one place, as notebook 1 runs it:

```
def grow(g):
  if stop(g): return leaf(g)
  gain = {c: ΔG(g, c) for c in splits(g)}
```

```
best = max(gain, key=gain.get)
if gain[best] <= 0: return leaf(g)
L, R = split(g, best)
return node(best, grow(L), grow(R))
```

- `g` is the group being grown; `stop(g)` is true at the depth limit, for a pure group, or with fewer than 2 examples.
- `splits(g)` lists every candidate, a feature with one of its midpoint thresholds; `gain` maps each candidate to its ΔG.
- `max(gain, key=gain.get)` returns the **candidate** with the largest gain, not the gain; `gain[best]` is the gain.
- If even the best gains nothing, the group becomes a leaf; otherwise it is split, and `grow` is called on each child.

What the two return, because the code hides it. `leaf(g)` is a new object that keeps one thing, the group's majority class, as its prediction. `node(best, grow(L), grow(R))` holds the question and its two subtrees. Python evaluates the arguments first, so `grow(L)` and `grow(R)` run all the way down before the node holding them is built: the tree is assembled from the leaves upward.

Training builds the tree; inference walks it. To predict for a new applicant, start at the root, go left if the applicant's value is at or below the threshold and right otherwise, until a leaf; return its class. The costs are lopsided: training scores every candidate at every node, while a prediction is at most `max_depth` comparisons, 8 for the best tree here, whatever the size of the training set. Lesson 6's k-nearest neighbours is the opposite: nothing to train, and every prediction compares the new point with all the training points.

Two predictable mistakes, both from reading too fast: "the function returns `g`" (it returns a leaf built from `g`), and "`max` gives the best gain" (it gives the best candidate).

Question — what does `node` do?

> *I follow the code, but I don't understand the function* ***`node`****.*

It builds a question node, an object holding three things: the question (feature and threshold) and the two subtrees. At the root that is `node((debt, 0.82), grow(the 971), grow(the 229))`. Because Python computes the arguments before calling the function, both subtrees are complete before the root node exists. In notebook 1 the same object is the class `Node`, with a feature, a threshold, a left and a right child, and a prediction for leaves; `predict_one` walks it.

Question — is training "building the tree" and inference "walking it"?

> *Can we say that for a decision tree, training is building the tree as described, and inference is sending a new sample down the tree according to its income and debt?*

Yes. What training learns is the questions at the nodes and the class at each leaf; that is the whole model. Inference is a handful of comparisons.

Slide 19 — Checked against scikit-learn

The from-scratch implementation of notebook 1 and scikit-learn's `DecisionTreeClassifier` make **the same prediction for all 1,200 applicants**, at every depth tested, and print the same root split. Not a coincidence: both search the same candidates by the same greedy rule. From here on the notebooks use scikit-learn's version: the same algorithm, compiled.

Question — why compare predictions, and on the training data?

> *Why predictions rather than accuracy? And why on the training data, when lesson 5 said never to judge a model there?*

Predictions, because two different trees can get the same **number** of applicants wrong while getting **different** applicants wrong; identical predictions, one by one, rule that out. On the training data, because this checks the **code**, not the model: the question is whether two programs make the same decisions from the same rows, and the rows they learned from are exactly the right ones to ask it on. No accuracy here is a claim about new applicants.

Slide 20 — What greedy misses

The search only ever asks "which single split helps most right now", never "which **pair** of splits helps most together". A feature that pays off only in combination with a later split can be invisible to a shallow tree.

That is what happens to the two interior islands. An island needs four cuts together, two on income and two on debt. Any one of them alone leaves both sides mixed, so its gain is small, and an easy strip always wins first. Slide 8's three rounds went to the strips.

Greedy is short-sighted, not blind. Each edge of an island pays a little on its own, so with enough depth the tree finds the islands one edge at a time: slide 8's third step already cuts the 884 at debt 0.404, next to island A's lower edge at 0.40, and by depth 8 both islands are drawn (slide 24). Greedy fails outright only where a single cut pays nothing at all, as in a chessboard or the exclusive-or (XOR) of two features, where every first cut leaves both sides exactly as mixed as before.

Question — is this why the islands are not found?

> *Greedy does not see the islands because it never considers whether one split combined with a later one could do better; it works one level at a time. Correct?*

Correct, and slide 24 shows the consequence of depth: at depth 2 no island, at depth 8 both.

Question — why can't the tree find the islands immediately?

> *If the islands are in the data, why does the tree not go for them first?*

Because identifying an island needs several coordinated thresholds, and greedy measures only the immediate gain of one. The first cut of an island is a weak split on its own; the strips are strong splits on their own. The tree takes the strong ones first and reaches the islands only when the strips are used up.

Slide 21 — Depth is the bias-variance dial

`max_depth` is the largest number of questions any one applicant answers. It is a tree's version of lesson 6's k:

- **A shallow tree** asks few questions and captures only the coarsest structure: high bias, low variance.
- **A deep tree** asks enough questions to isolate single points: training accuracy **exactly 1.000**, even on data with no structure at all, unless two examples share every feature value but not the label.

Two pictures make the dial concrete, both from notebook 1's depth table.

- **Who votes in a leaf.** A leaf predicts the majority of its applicants. At depth 2 there are 4 leaves, with a median of 123.5 applicants each: a vote of a hundred. The unconstrained tree has 156 leaves with a median of 2, and 72 of them hold a single applicant: a prediction made from one person's label, lesson 6's $k = 1$.
- **The line at 0.929.** The rule that generated the data scores 0.929 on these labels, because 85 were flipped. A training accuracy above that line means the tree agrees with the labels better than the truth does, so it must be reproducing flipped labels: depth 8 is just past it at 0.951, the unconstrained tree has learned all 85.

An image for it: a checklist of three questions written from last year's customers works on this year's; fifty questions, each written about one past customer, do not.

Question — is that the whole picture?

> *A shallow tree: few questions, bias, underfitting. A deep tree: many questions, it isolates single points and "learns" those data, overfitting. Correct? What other intuition could I give?*

Correct, with the pair completed: shallow is high bias and **low variance**; deep is low bias and **high variance**. High variance means that trained on another sample of applicants, a deep tree comes out very different: slide 26 measures it. The two pictures above, who votes in a leaf and the 0.929 line, are the intuitions that land.

Question — why can an unconstrained tree reach 1.000 on noisy data?

> *The labels are 7% noise. How can any model be right on all of them?*

By memorising them. Splitting need not stop until every leaf is pure, at worst one example per leaf, which the tree then "predicts" perfectly by definition. The only exception is two applicants with identical income and debt but different labels, which no split can separate; with two continuous features drawn at random, that does not happen. A score of 1.000 shows the model can memorise, not that it has learned.

Slide 22 — Measured, at seven depths

`max_depth`	Training accuracy	Cross-validated (mean ± standard deviation)
2	0.840	0.836 ± 0.024
4	0.843	0.827 ± 0.025
6	0.898	0.866 ± 0.016
8	0.951	**0.882 ± 0.010**
12	0.986	0.862 ± 0.018
16	0.993	0.852 ± 0.021
unconstrained	**1.000**	0.852 ± 0.021

Read the two columns against each other, as lesson 5 taught. Training accuracy climbs towards 1.000; cross-validated accuracy rises, peaks at depth 8, and falls back, even though the model keeps getting more flexible. Chosen by the training column, the unconstrained tree would look unbeatable; it is in fact 3 points below the depth-8 peak.

Question — how is the cross-validated accuracy computed, and what is the ±?

> *Remind me how the cross-validated accuracy is evaluated. There is also a range: how is it obtained?*

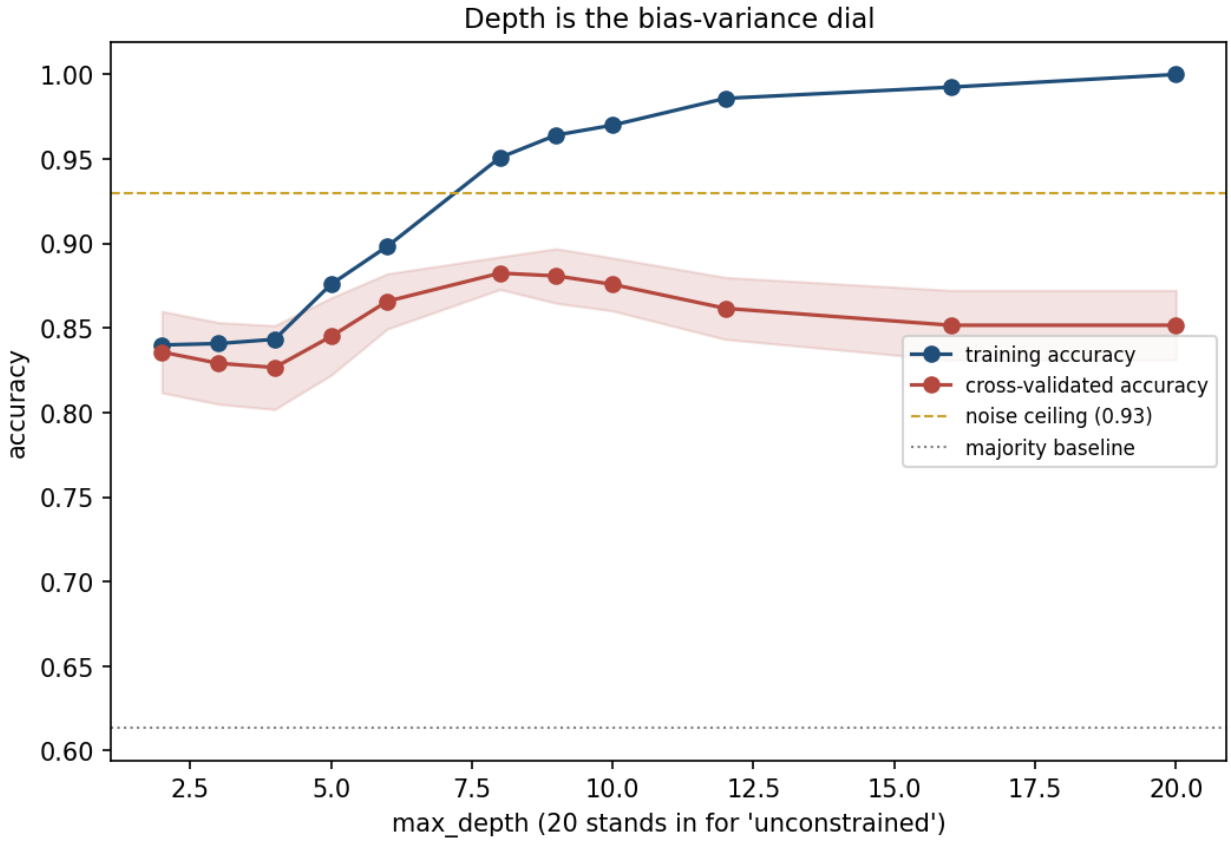
Five-fold stratified cross-validation, lesson 5's procedure. The 1,200 applicants are cut into five folds of 240, each with the same share of defaulters. Five times, the tree trains on 960 and is scored on the 240 it did not see. At depth 8 the five scores are 210, 211, 215, 214 and 209 correct of 240: 0.875, 0.879, 0.896, 0.892 and 0.871. The table shows their mean, $(0.875+0.879+0.896+0.892+0.871)/5 = 0.882$, and after ± their standard deviation, 0.010: how much the score moves from one fold of 240 to another. It is not a confidence interval.

Question — why is depth 4 worse than depth 2?

> *Shouldn't a deeper tree always do at least as well on new data, up to the peak?*

No: more flexibility does not guarantee a better score on unseen rows. The two extra levels between depth 2 and depth 4 still cannot reach the islands (slide 20), and the splits they do make are partly fitted to details of the training folds that do not repeat in the held-out fold. The difference, 0.836 against 0.827, is also well inside one standard deviation: on these five folds the two are effectively level.

Slide 23 — The curve bends downward



Training accuracy (blue) and cross-validated accuracy (rust) against depth, with the noise ceiling dashed in gold and ± one standard deviation shaded.

The table as a curve, with more depths. Training climbs steadily; cross-validated accuracy rises and then turns over. The peak is where the tree is deep enough to isolate both islands and shallow enough to leave the flipped labels as errors rather than as leaves of their own. Past it, more and more of each extra split goes on fitting the 7% of flipped labels, and a flipped label has no pattern to learn.

Two reading details: the plot has eleven depths where the table had seven, and "20" on the axis stands for the unconstrained tree, whose real depth is 24.

The shape is not new: lesson 3's test error turned back up as the polynomial degree grew, and lesson 6's k-nearest neighbours did the same as k shrank towards 1.

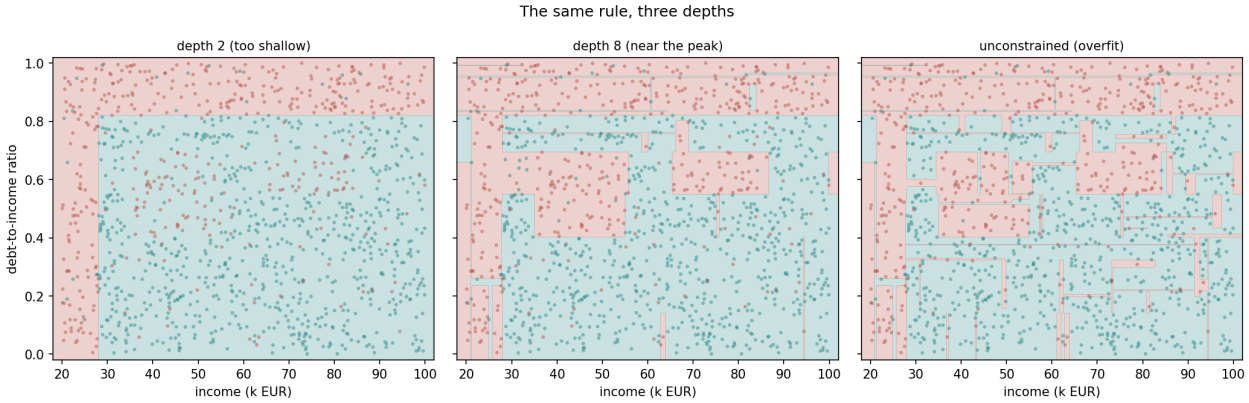
Question — is the curve just the previous table?

> *Training always rising, cross-validated with a maximum, then the noise ceiling: with a certain number of flipped labels it is the best a model could reach; oddly, the training accuracy goes beyond it, which would need justifying. At the bottom the baseline. Correct?*

Correct (the baseline is 0.613). The crossing is the point to justify. The ceiling is what a model can expect, on average, on **new** applicants, whose flipped labels it cannot know. On its own training data it can do far better, by memorising them: the blue curve crosses the gold line just before depth

8, where memorising starts, and at 1.000 all 85 flipped labels are learned. The rust curve does not cross. On a test set a score a little above 0.93 would be possible by luck (slide 4), but not a score consistently far above it.

Slide 24 — The same rule, three depths



The boundary a tree draws at depth 2, depth 8 and with no limit, over the 1,200 applicants.

- **Depth 2.** The income floor and the debt ceiling, the two biggest and easiest regions; neither island is visible. The tree asks three questions, but the third splits the high-debt group into two groups both predicted "default", so it draws no line.
- **Depth 8.** Both islands appear, close to the true rectangles though not exact, and a few thin slivers have already started: its training accuracy, 0.951, is past the 0.929 line.
- **No limit.** A fringe of tiny rectangles chasing individual flipped labels, in both colours: rust slivers in healthy territory, and teal ones inside the risky strips, where a flipped "repaid" sits. Slide 8's applicant at income 23.22 and debt 0.9475 is one, in the top-left corner.

The full-depth boundary is not a smoother version of the depth-8 one. It is a noisier one, built by the same mechanism that gave the unconstrained tree its perfect training score.

Question — does it all add up?

> *Depth 2: high bias, low variance. Unconstrained: overfitting, every training point memorised, small red islands around them. Depth 8: the right compromise, it finds low income, high debt and the two islands. Does it all add up?*

Yes, with three precisions. The memorised fringe at full depth goes both ways: red slivers where defaulters sit in healthy territory, teal ones where flipped "repaid" labels sit inside the strips. Depth 8 has already started to memorise, a few thin slivers, as its 0.951 past the 0.929 line says: the best compromise, not a clean one. And depth 8's islands are close to the true rectangles, not exact.

Question — why rectangles and not smooth curves?

> *Why does a deeper tree draw more and more little rectangles instead of a smoother boundary?*

Because every split is a threshold on one feature, a vertical or horizontal cut, so every region a tree draws is a rectangle. More splits make smaller rectangles, never curved ones; at full depth some of them enclose single noisy applicants.

Slide 25 — Reading a tree, and the knob that keeps it small

Two knobs limit a tree, and they limit different things.

- **`max_depth` caps every path.** Inside the cap, every node that can be split is split, whether or not it is worth it: slide 24's depth-2 tree spent one of its three questions on a split that changed no prediction.
- **`max_leaf_nodes` is a budget of leaves**, spent one at a time, **best first**: at each step, among all the current leaves, the one whose split gains most, with each leaf's gain ΔG **weighted by its share of all the applicants**. It is the one place where gains at different nodes are compared, which slide 18's loop never does.

With a budget of nine leaves the tree scores **0.908** on its training data and **0.901** cross-validated, above the depth-8 tree's 0.882 with its 71 leaves, and its thresholds are recognisable: 28 for the income floor, 0.82 for the debt ceiling. `export_text(tree)` prints it as a few lines of if/else rules: that printout is the whole model, and a loan officer could apply it by hand. It is what k-nearest neighbours and support vector machines cannot offer at all, and for a decision that must be explained to the person it affects, that can matter as much as accuracy.

Why does the small tree beat the bigger one? This dataset's rule is made of axis-aligned rectangles, exactly the shape a few well-placed thresholds draw. The 9 was chosen for readability, not by trying budgets on the folds.

A predictable mistake. "The leaf with the largest gain is split next." That is the rule of slide 18 carried over, and it is natural. But best-first growth compares gains from groups of different sizes, so each gain is multiplied by its group's share of the applicants first. A moderate gain on a large group beats a large gain on a tiny one.

Question — is 90.8% the tree's accuracy?

> *The nine-leaf tree "brought the accuracy to 90.8%"; export_text prints it as if/else; a single tree is very readable, an ensemble loses that. Correct?*

Correct on readability, with one precision: 0.908 is the **training** accuracy, which cannot be set against the cross-validated 0.882 of slide 22. The comparable number is the cross-validated 0.901.

Question — how can nine leaves beat seventy-one?

> *How can a tree with nine leaves do better on new data than one with 71?*

The budget spends its nine leaves where they gain most, the strips and the islands. The depth-8 tree also spends splits on small groups with weak gains, which improve its training fit without improving its predictions on new applicants. On rectangle-shaped data, nine well-placed thresholds are close to all there is to learn.

Worked example — which leaf a budget of leaves splits next

Two leaves of a tree trained on 1,200 applicants could be split next. Leaf B's best split has the larger gain inside its own group; leaf A's group is much larger.

leaf	applicants	best ΔG in the leaf	share of all applicants	weighted gain
A	900	0.020	$900 \div 1{,}200 = 0.750$	$0.020 \times 0.750 = 0.0150$
B	30	0.300	$30 \div 1{,}200 = 0.025$	$0.300 \times 0.025 = 0.0075$

`max_leaf_nodes` spends the next leaf on **A**: its gain is smaller, but it improves the purity of 75% of the applicants, against 2.5% for B. Judged by the local gain alone, B would win; that is the slip the slide warns about.

Slide 26 — A tree that doesn't stay still

The instability, measured. Two trees at `max_depth = 6`, each trained on 780 of the 1,200 applicants drawn **without replacement** (no applicant twice; the two samples share 499).

- **The root barely moves**: 0.820 against 0.819, the debt ceiling both times. Inside each sample the ceiling separates 636 applicants at about 25% default from 144 at 94%, and a contrast that size does not change from one sample to the next.
- **Below the root, the trees differ**: 31 and 33 leaves, and **opposite verdicts for 140 of the 1,200 applicants**, 11.7%.

The root is decided on all 780 applicants with overwhelming evidence; the 29 or so questions below it are each decided inside smaller groups, where a few different applicants change the winner. And those questions are most of the tree: 31 leaves mean 30 questions, one at the root and 29 below it.

Question — what does "without replacement" mean?

What does "no replacement" mean, and why does it matter here?

Each of the 780 is a different applicant, like cards dealt without putting them back. With replacement, which the next part of the lesson uses, a card goes back in the deck and can come out twice. Here the point is only that the two trees saw two different, overlapping groups of real applicants.

Question — where do 26% and 94% come from?

Why do the two roots agree so well?

The two roots agree because inside each sample of 780 the debt ceiling separates groups as different as 24.5% default against 93.8% (sample A) and 24.8% against 93.8% (sample B), with 636 and 144 applicants: the same contrast in both. A split deep in the tree is decided by a few dozen applicants instead.

Question — why are the two thresholds different at all?

> *0.820 and 0.819: shouldn't the same question give the same threshold?*

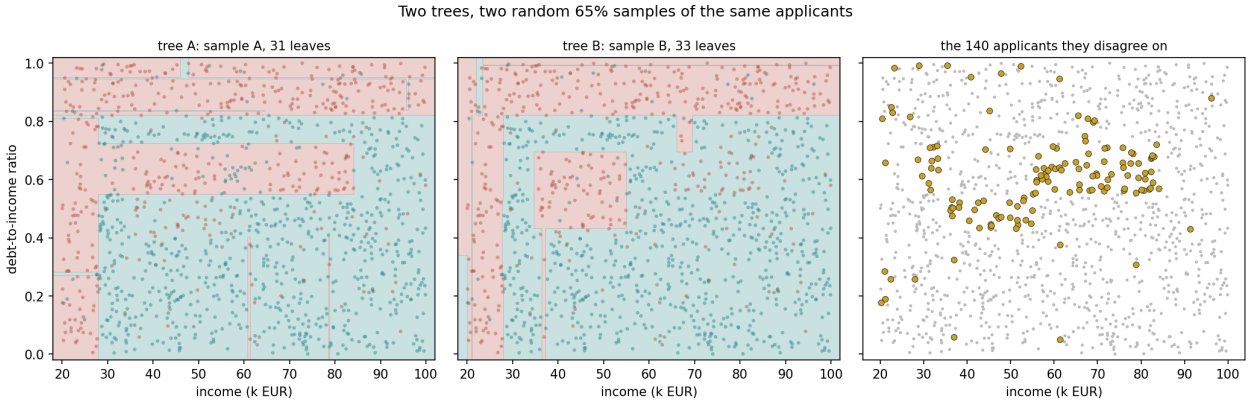
Same question, different midpoint. The threshold is the midpoint between the two neighbouring values **present in the sample** (slide 14). Sample A holds applicants at 0.8198 and 0.8205: midpoint 0.82015. Sample B lacks the one at 0.8198, so its neighbours are 0.8182 and 0.8205: midpoint 0.81935. Both send every applicant of their own sample the same way the true ceiling would; the difference is which applicant happened to be drawn.

Question — "strongest" and "weaker" splits?

> *"The strongest split is stable, the weaker ones are not": strongest is the first split, weaker are the ones below, where one applicant more or less makes the difference. Correct?*

Correct. "Strongest" means the largest gain with the most evidence behind it, which greedy takes first, so here it is the root. "Weaker" means smaller gains decided inside smaller groups.

Slide 27 — Two trees, 140 different verdicts



Left and middle: the two trees' boundaries. Right: the 140 applicants they give opposite verdicts to, in gold.

The previous slide's two trees, drawn. The strips agree: both trees find the income floor and the debt ceiling. The interior does not: tree A draws one long band across both islands, tree B finds island A and misses island B.

Where the gold points sit is the lesson of the slide. Three in four of them, 105 of 140, are in or right next to the two islands (75 inside them), where each split was decided by a few dozen applicants. Of the other 35, 19 lie in the two strips (11 below the income floor, 11 above the debt ceiling, 3 of them in the corner where both apply) and 16 elsewhere. That is a tree's variance, made visible: for 140 people, the verdict depends on which sample the bank happened to train on.

A predictable mistake. "The trees disagree on 11.7%, so each tree is wrong on 11.7%." Reasonable, since a disagreement and an error are both a verdict that differs. But a disagreement compares the two trees with each other, not with the labels: on each of the 140, exactly one of the two is right, and elsewhere both can be right or both wrong. Disagreement measures **instability**; accuracy needs the labels.

Question — are the 140 in the islands?

> *The 140 applicants classified differently sit in the two central islands: it is variance made visible. Correct?*

Mostly. Counted against the true rectangles, three in four are inside or right next to the islands; the rest are in the strips or elsewhere. "Almost all in the islands" would overstate it.

Slide 28 — Notebook 1, live

The notebook's map: each section, the slides it belongs to, the calls it makes and the number to come back with. Section 1 compares logistic regression with the baseline (0.748 against 0.613); section 2 builds a tree from scratch and checks it against `DecisionTreeClassifier` (identical predictions, root gain 0.1438); section 3 sweeps the depth (peak 0.882 at depth 8); section 4 prints the nine-leaf tree and measures two trees' disagreement (140 opposite verdicts).

The cell worth protecting is the last one: seeing the 140 appear on your own screen lands harder than reading it off a slide. The consolidation notebook `Consolidation/07_lesson_7_atoms.ipynb` has short predict-then-run exercises on the same ideas: Gini by hand, the root gain from counts, thresholds as midpoints, a depth-2 tree read with `export_text`.

Slide 29 — Break

Twelve minutes. What comes next starts from the last two slides: one tree is unstable, and the rest of the lesson turns that weakness into the ensemble's strength.

Part II — Many trees that vote

Slide 30 — Bagging: many trees, one vote

The first remedy. Slide 27 showed two trees, trained on samples of the same applicants, disagreeing on 140 of 1,200. If those disagreements come from the luck of each sample, then many trees, each trained on its own sample and combined by a **vote**, should cancel them.

The samples are **bootstrap samples**: m rows drawn **with replacement** from the m training rows, so some rows come out twice and some never. Every tree sees as many rows as a single tree would, and every sample is different. A tree on each sample, then a majority vote: that is **bagging**, from **b**ootstrap **agg**regat**ing**. In this lesson the trees are fully grown, but bagging works with any tree, or any model.

Why a vote cancels the differences, one applicant at a time. A tree's verdict on an applicant depends on two things: what the data really say in that part of the plane, which is the same for every tree, and the luck of its own bootstrap sample, which is different for every tree. A mistake caused by luck falls on different applicants for different trees, so on any one applicant the unlucky trees are a minority, and the vote outvotes them. In notebook 2, on its 360 test applicants: the applicant at income 39.22 and debt ratio 0.9721, who defaulted, gets "default" from 69 of the 100

trees and "repaid" from 31; the vote is right. Over all 360, one tree alone is wrong on 58.7 on average; the vote gets 18.9 of those right; the vote itself is wrong on 46.

A predictable mistake. "The trees cancel each other's errors." Reasonable, because slide 27 is all disagreement. But the vote cancels only the mistakes that **differ** from tree to tree. A flipped label is not one of them: on a new applicant whose label was flipped, the trees have learned the rule that generated the data from the applicant's neighbours, and on average 95 trees in 100 vote what the rule says. Of the 29 flipped labels among notebook 2's 360 test applicants, the vote gets 1 right.

The trees are independent of one another, so they can be grown in parallel (`n_jobs`). scikit-learn's default is `n_estimators=10`; notebook 2 uses 100. scikit-learn averages the trees' predicted probabilities rather than counting votes; with fully grown trees every leaf is pure, every probability is 0 or 1, and the two rules give the same answer.

A note of history. Bagging is Leo Breiman's, published in 1996 under the title "Bagging predictors": the same Breiman who co-wrote the CART trees of 1984 and, five years after bagging, the random forest.

Question — could the trees not "vote both ways" on a flipped label and so get it right?

> *Make clear that only the disagreements cancel. And for the flipped labels: could the trees not predict opposite things and so reach the correct prediction?*

No. The vote gives one answer per applicant, so for a flipped applicant to come out right, the **majority** of the trees must vote against the rule that generated the data at that point. On new data no tree has seen that applicant; its neighbours follow the rule, so nearly every tree votes the rule. There is nothing to cancel: the trees agree, on the wrong answer. And the flip is random, unrelated to income and debt, so the features carry no clue about who was flipped.

On the **training** data the picture is different, and instructive: there, the trees whose sample contains a flipped applicant have memorised it, and the vote reproduces the flipped label on 55 of the 56 in notebook 2's training split. Either way it is agreement, not disagreement, and agreement is what a vote cannot cancel.

Question — is bagging a trick to force disagreements?

> *So bagging is a "trick" to make the trees see different combinations, forcing disagreements that the vote then cancels?*

Almost. Bagging does not **create** the disagreements: a single tree already has them, unseen, as slides 26 and 27 showed with another sample. The bootstrap imitates collecting the loans again, without new data; each tree answers as it would on another plausible dataset; the vote averages over them. The luck cancels; what the data really say stays.

One consequence follows: bagging helps only **unstable** models. On the same split, a depth-2 tree scores 0.806, and a bagging of 100 depth-2 trees also 0.806: those trees barely differ, so there is nothing to average. The unconstrained tree goes from 0.825 to 0.872.

Question — is bagging to a deep tree what cross-validation is to logistic regression?

> *Could we say that bagging is to deep trees what cross-validation is to logistic regression?*

No, though the two share an ingredient. Cross-validation **measures**: it estimates how a model does on new data, and leaves the model unchanged. Bagging **builds**: it makes a new, steadier model. Both resample the data to deal with the luck of the sample: cross-validation measures how much a result depends on it, bagging reduces that dependence.

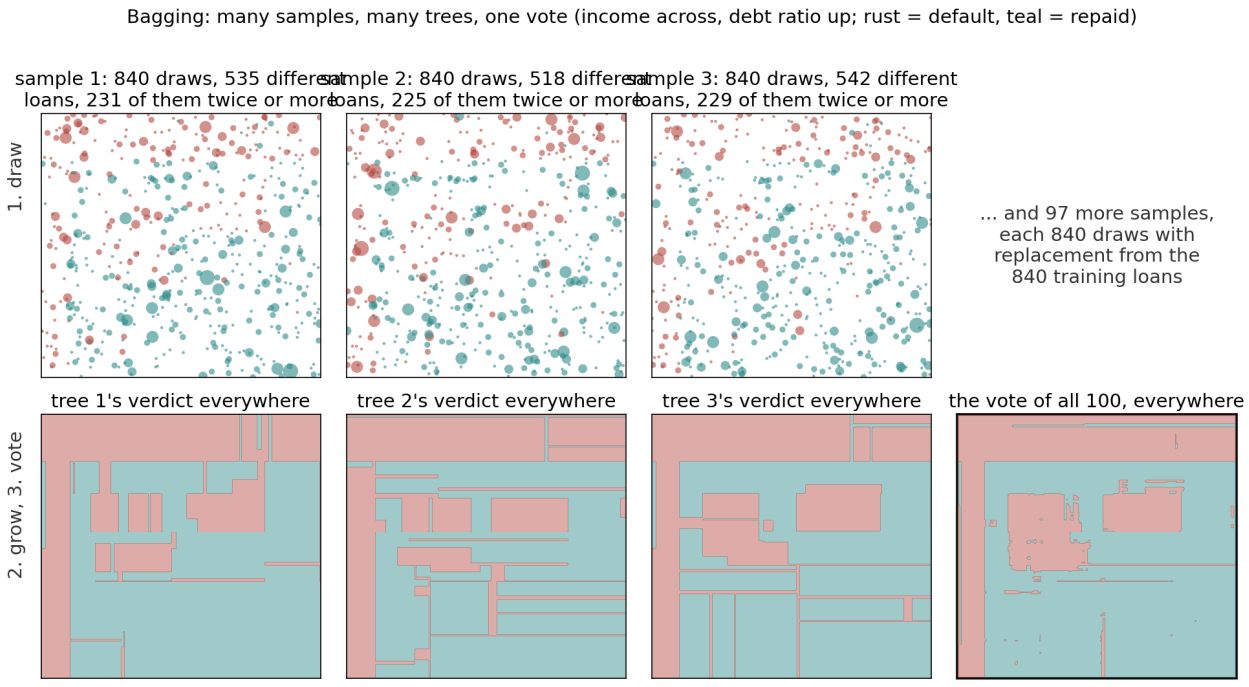
The closer analogy is Ridge on lesson 3's degree-12 polynomial: a flexible, unstable model and a remedy for its variance, Ridge by restraining the coefficients and bagging by averaging many models. Neither touches the bias.

Worked example — five trees vote on three applicants

applicant	trees 1 to 5	count	vote	label	
P	D D R D D	4 D, 1 R	D	D	right
Q	R D R R R	1 D, 4 R	R	R	right
S	R R R R R	0 D, 5 R	R	D	wrong

On P and Q one tree each is wrong, a different tree each time: they are outvoted. On S every tree is wrong the same way, because S's label was flipped and its neighbours follow the rule; there is no minority to outvote. Each tree alone is wrong on 1.4 of the three applicants on average; the vote is wrong on 1. The vote removes the mistakes that differ, not the one they share.

Slide 31 — Bagging, drawn



Top: three bootstrap samples of the 840 training loans (a dot is bigger the more times that loan was drawn; loans never drawn are absent). Bottom: the decision map of the tree grown on each sample, and of the vote of all 100.

The previous slide as one picture, from notebook 2. A bagging of 100 trees is trained on the 840 training loans of a 70/30 split. The top row shows what the first three trees' samples drew: every sample is 840 draws, and the number of **different** loans varies by luck around $840 \times 0.632 = 531$: 535, 518 and 542 here. The bottom row asks each tree about a dense grid of imaginary applicants covering the whole plane, and colours each point by the tree's verdict: the tree's decision map. The last panel is the same map for the vote.

Compare the three trees first: three samples of the same 840 loans, three different shapes, which is slide 27 again, a hundred times over. Then the vote: the two strips and the two islands survive, and most single-tree slivers do not, because a sliver that one tree draws, most of the others do not.

Question — why do the samples have different sizes?

Why do the three samples have 535, 518 and 542 loans?

They do not: every sample is 840 draws. 535, 518 and 542 are the **different** loans drawn; the rest of the 840 draws are repeats. Sample 1, for instance, drew 535 different loans, 231 of them twice or more, and missed the other 305 of the 840.

Question — does the bottom row show predictions on the training set?

So the bottom row contains the trees' predictions on the training data?

No: on the whole plane. Each tree, trained on its sample, is asked about a grid of imaginary applicants covering income 20 to 100 and debt 0 to 1, and every point is coloured by its verdict. It is the same technique as slide 24's figure: a picture of the rule the tree has learned, not of its training loans.

Slide 32 — How much data one bootstrap sample sees

One draw picks a given row with probability $1/m$, so it misses it with $1 - 1/m$. All m independent draws miss it with $(1-1/m)^m$: at $m = 1{,}200$ that is 0.3677, and as m grows it tends to $e^{-1} = 0.368$. So each sample holds about **63.2%** of the different rows; the other **36.8%** are that tree's **out-of-bag (OOB)** rows, which it never sees.

The sample still has m draws. At $m = 1{,}200$, on average $1{,}200 \times (1 - 0.3677) = 759$ different rows, and the other 441 draws pick a row already drawn.

A predictable mistake. "Each tree is trained on 63.2% of the data, 759 rows." It is the number on the slide, so it is natural. But the tree sees 1,200 rows: 759 different ones, some of them twice or three times.

A second one. "The out-of-bag rows are left out of the training set." The word "out" makes them sound like a test set. But out-of-bag is decided **tree by tree**: the row is missing from this tree's sample, and the other trees, about 63% of them, do see it. It stays a training row.

One identity is worth seeing. The 1,200 draws are the different rows plus the repeated draws; the 1,200 rows are the different rows plus the rows never drawn. So the repeated draws are exactly as many as the out-of-bag rows: every repeat takes the place of a row left out. The rows drawn twice or more are fewer, about $1{,}200 \times 0.264 = 317$, because some take three or four draws.

Question — 441 repeated?

> *63.2% of the distinct rows are drawn, the rest stay out-of-bag; of 1,200 applicants, 759 distinct are drawn and 441 are repeated. Correct?*

Correct except for "441 repeated": 441 are repeated **draws**, 1,200 − 759. The applicants drawn two or more times are about 317, because some of them take three or four of those draws. And 759 and 441 are expected counts, averages over many samples.

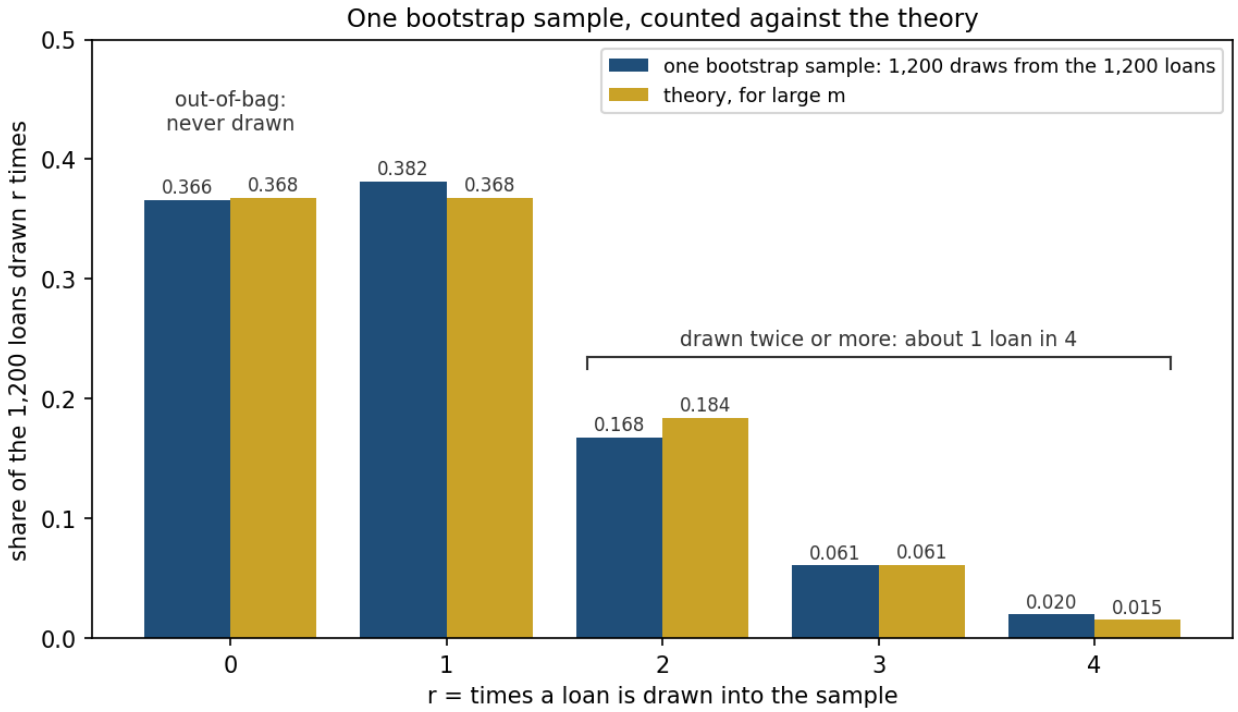
Worked example — one bootstrap sample of five rows

Five training rows, numbered 1 to 5; five draws with replacement: 2, 4, 2, 1, 2.

- Times each row is drawn: row 1: 1, row 2: 3, row 3: 0, row 4: 1, row 5: 0.
- Different rows drawn: 3. Never drawn, the out-of-bag rows of this tree: rows 3 and 5.
- Repeated draws: $5 - 3 = 2$, exactly as many as the rows never drawn: each repeat took the place of a row left out.

The theory for $m = 5$: a row is missed by all five draws with probability $(1 - 1/5)^5 = 0.328$, so on average 1.64 rows of 5 are out of bag. For large m the share tends to 0.368.

Slide 33 — One bootstrap sample, counted against the theory



Each pair of bars is one value of r, the number of times a loan is drawn into the sample; a bar's height is the share of the 1,200 loans drawn exactly r times. Blue: one real sample. Gold: the theory for large m.

The $r = 0$ bars are the loans never drawn, the out-of-bag ones: 0.366 in this sample, 0.368 in theory. A bar times 1,200 is a count: 0.168 at $r = 2$ is about 202 loans drawn exactly twice. One real

sample already behaves like the theory, and about **one loan in four** is drawn two or more times. Every tree sees a different mix of repeats and absences, and that is where their disagreement comes from.

The gold bars come from counting. A loan is drawn exactly r times when r of the m draws pick it and the other $m-r$ miss it: $\binom{m}{r}$ ways to choose which draws, each with probability $(1/m)^r(1-1/m)^{m-r}$. For large m this tends to $e^{-1}/r!$, the Poisson distribution with mean 1. Handout section 5.1 has the two limits step by step.

Question — why are "never" and "once" almost equally likely?

Why are the bars for r = 0 and r = 1 almost the same height?

Think of 1,200 balls thrown at random into 1,200 boxes: on average one ball per box, and about as many empty boxes as boxes with exactly one ball. In three steps:

1. **Never.** Every one of the 1,200 draws must miss the loan: $(1 - 1/1{,}200)^{1{,}200} = 0.3677$.
2. **Exactly once.** Draw number 1 picks it and the other 1,199 miss it, $(1/1{,}200)(1-1/1{,}200)^{1{,}199}$; but the draw that picks it can be any of the 1,200, so there are 1,200 such cases, and $1{,}200 \times 1/1{,}200 = 1$. What is left is $(1 - 1/1{,}200)^{1{,}199} = 0.3680$.
3. **The comparison.** The two formulas differ only in the exponent, one factor $1 - 1/1{,}200 = 0.9992$.

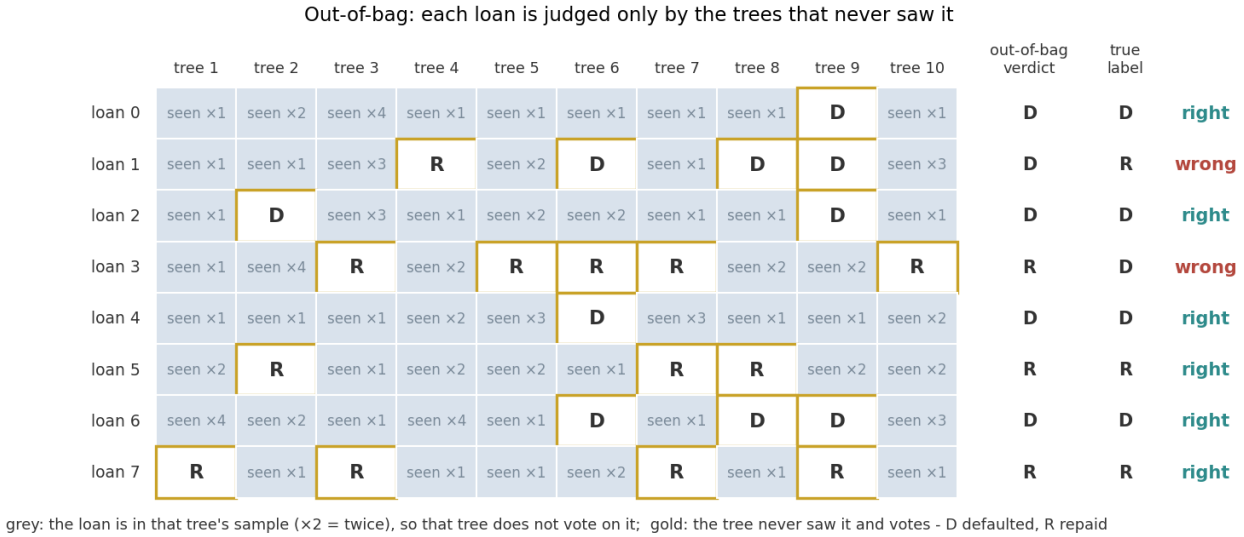
The reason underneath: each loan is drawn once on average. With 2,400 draws, "once" would be twice as likely as "never".

Question — what is on the axes?

What exactly is on the x axis, and on the y axis?

On the x axis, r: how many times a loan is drawn into the sample, 0 to 4. On the y axis, the share of the 1,200 loans drawn exactly that many times. The figure reads "for each count, how many loans", not "for each loan, how many times".

Slide 34 — Out-of-bag: each loan judged by the trees that never saw it



Ten bagged trees trained on all 1,200 loans; the first eight loans. Grey: the loan is in that tree's sample, so the tree does not vote on it. Gold: the tree never saw the loan, and its vote is written in, D for defaulted, R for repaid.

Read one row before the rule. A grey cell is a tree that has already seen the loan, like a student who has read the answer, so it does not vote ("seen ×2" means drawn twice). A gold cell is a tree that never saw it, and it votes. The **out-of-bag verdict** is the majority of the gold votes, compared with the true label.

- **Loan 0.** Only tree 9 never saw it; it votes D; the loan defaulted: right.
- **Loan 3.** Trees 3, 5, 6, 7 and 10 never saw it and all vote R, but it defaulted: it sits inside the second island, which those trees missed.

Do this for all 1,200 loans and count how often the verdict is right: that share is the **out-of-bag accuracy**. Every loan is judged only by trees for which it is unseen data, so it works like a validation set, with nothing set aside and nothing trained again. A tie, if one happens, goes to "repaid": scikit-learn takes the class with the highest average probability, and an exact tie goes to the first class.

With ten trees, about $0.368 \times 10 = 3.7$ vote on each loan, and a loan can sit in all ten samples and get no vote at all: the chance is $0.632^{10} = 0.0102$, about 12 of the 1,200. The grid illustrates the rule; the real computation, on the next slide, uses 300 trees.

Question — what if a loan is in every tree's sample?

What happens to a loan that every tree has seen?

It has no out-of-bag voter, so it gets no out-of-bag prediction. With ten trees that happens to about 12 loans in 1,200; with 300 trees the chance, 0.632^{300}, is negligible. That is one reason the out-of-bag score is computed on a large ensemble.

Slide 35 — Out-of-bag against cross-validation

How accurate is a bagging of 300 trees on loans **it never saw**? All 1,200 loans train it, and two methods answer.

- **Out-of-bag.** Each loan is judged by the trees that never saw it: 110.5 of the 300 on average ($0.368 \times 300 = 110$), from 81 to 133, a different set for each loan. **1,090** of the 1,200 are right: accuracy **0.9083**. The trained bagging already contains everything needed; scikit-learn reports it as `oob_score_`.
- **Five-fold cross-validation**, lesson 5's method. Five more baggings, each trained on 960 loans and scored on the other 240: **1,083** right, accuracy **0.9025 ± 0.021**.

They agree within 0.006, well inside the ± of 0.021. That is why the comparison matters: cross-validation is lesson 5's trusted check, and out-of-bag nearly matches it for free. Do not read it to the fourth decimal: over twelve different seeds the gap between the two averages 0.0040 and is at worst 0.0075.

Question — is the area under the curve evaluated?

Is the area under the curve (AUC) evaluated here? The problem is a classification one.

No. Both numbers are **accuracies**: the share of loans whose default-or-repaid verdict is right.

Question — are the two methods the same thing?

If both estimate accuracy on unseen loans, is out-of-bag just cross-validation in disguise?

They estimate the same thing with different arrangements, so their agreement is reassuring, not guaranteed. Out-of-bag judges each loan with about 110 trees, from an ensemble trained on all 1,200 loans; cross-validation judges each group of 240 with all 300 trees of an ensemble trained on 960. On this data the two land within their own noise. The out-of-bag score asks no extra training, which is its whole appeal when data or time are short.

Slide 36 — Cross-validation, step by step: five groups of 240

The previous slide's ± 0.021, computed. The 1,200 loans are split into 5 groups of 240. In each round a bagging of 300 trees is trained on the other four groups, 960 loans, and its verdicts are counted on the group left out. Every loan is judged once, by a bagging that never saw it.

Round	Right of its 240	Accuracy = right ÷ 240	Deviation = accuracy − 0.9025	Deviation^2
1	212	0.8833	−0.0192	0.000367
2	215	0.8958	−0.0067	0.000044
3	223	0.9292	+0.0267	0.000711
4	222	0.9250	+0.0225	0.000506
5	211	0.8792	−0.0233	0.000544

The mean of the five accuracies is 0.9025, equal to 1,083/1,200 because the groups are the same size. The standard deviation: the squared deviations sum to 0.002174; divided by 5, square root, 0.0208, printed 0.021. A detail for anyone redoing it: numpy's `np.std` divides by 5; some statistics texts divide by 4, which gives 0.023.

A predictable mistake. Reading the deviation column as "out-of-bag minus cross-validation", because the previous slide compared the two. This table is cross-validation alone; out-of-bag plays no part in it. The deviation is each round's accuracy minus the mean of the five rounds.

Question — why divide by five and not four?

Statistics courses divide by $n - 1$. Why five?

Because `np.std`, which scikit-learn's examples use, divides by the number of values by default. Dividing by four estimates the spread of the population the folds came from, and gives 0.023. For comparing models on the same folds either convention works, as long as it is the same for all of them.

Slide 37 — Why voting helps, and its floor

The intuition behind every number so far, in four steps.

1. A tree's verdict has **two parts**: what the 1,200 loans say, shared by all the trees, and the **luck of its own bootstrap sample**, private to it.
2. The luck differs from tree to tree, so in a vote it **cancels**.
3. If the 1,200 loans themselves mislead, every tree is misled **alike**, and the vote **cannot cancel** that.
4. So more trees help only down to a **floor**; to go lower, the trees must be **less alike**.

An image for it: ten experts who all read the same newspaper. Their personal opinions differ and cancel out in a vote; a mistake printed in the newspaper, all ten repeat.

The same reason explains what bagging cannot fix: a mistake every tree makes, such as the flipped labels of slide 30 or a tree too shallow to draw the rule that generated the data, is shared, so it stays. Handout section 5.2 has the formula behind these words, the variance of an average of B correlated verdicts, $\sigma^2/B + \rho\sigma^2(B-1)/B$, which tends to $\rho\sigma^2$: the floor.

Question — how can the 1,200 loans "say something wrong"?

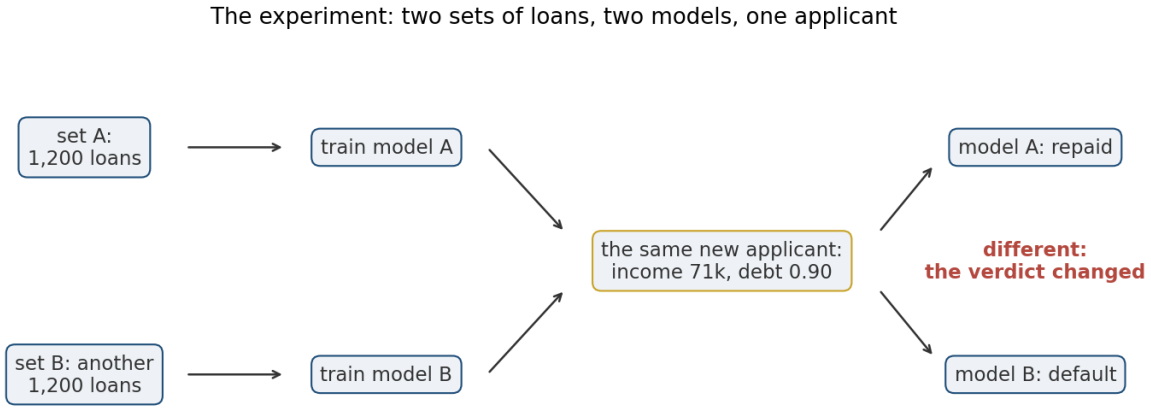
How does it happen that the 1,200 loans mislead the trees?

They are one sample of the lender's applicants, and a sample's chance features cannot be told apart from the rule that generated the data. Three ways, all seen today:

- **Zones with few applicants.** Island A's lower edge was found at 0.404, not 0.40 (slide 20). Every bagged tree draws from the same 1,200, so every tree puts it there.
- **Flipped labels that happen to fall close together** look like a small risky pocket. Each bootstrap sample holds about 63% of them, so many trees learn the pocket, and the vote confirms it.
- **Thresholds set by the neighbours present**: 0.82015 for a ceiling of 0.82.

The only remedy for these is more data. The next two slides measure how much of each tree's instability is of this shared kind.

Slide 38 — The experiment: two sets of loans, two models, one applicant



Repeat for 300 applicants and for every pair of the 40 sets of loans: on what share of applicants do the two models disagree?

The experiment of the next slide, drawn: two sets of 1,200 loans from the same lender, the same model trained on each, both asked about the same applicant.

Before the curve, the experiment it comes from, in five steps:

1. The generator produces a set A of 1,200 loans and a set B of another 1,200 from the same lender, as if the bank had collected its data in two different years.
2. The same model is trained on each: model A and model B.
3. Both are asked about the **same new applicant**.
4. If one says "default" and the other "repaid", the verdict **changed**: for that person, the answer depends on which year's loans the bank happened to collect.
5. Repeat for 300 applicants spread over the plane and for every pair of 40 sets of loans, and count the share of applicants whose verdict changes.

The applicant drawn, at income 71 thousand euros and debt ratio 0.90, is a real case from notebook 2: one tree trained on set A says "repaid", one trained on set B says "default"; the votes of 99 trees on the two sets both say "default".

This measures **stability, not correctness**: it does not ask whether a verdict is right, only whether it stays the same. And it is possible only because these data are synthetic. A real lender has one set of loans, which is exactly why the bootstrap imitates other collections by resampling the one it has.

Question — changes with respect to what?

> *Another 1,200 applicants: what does "the verdict changes" mean? Changes with respect to what?*

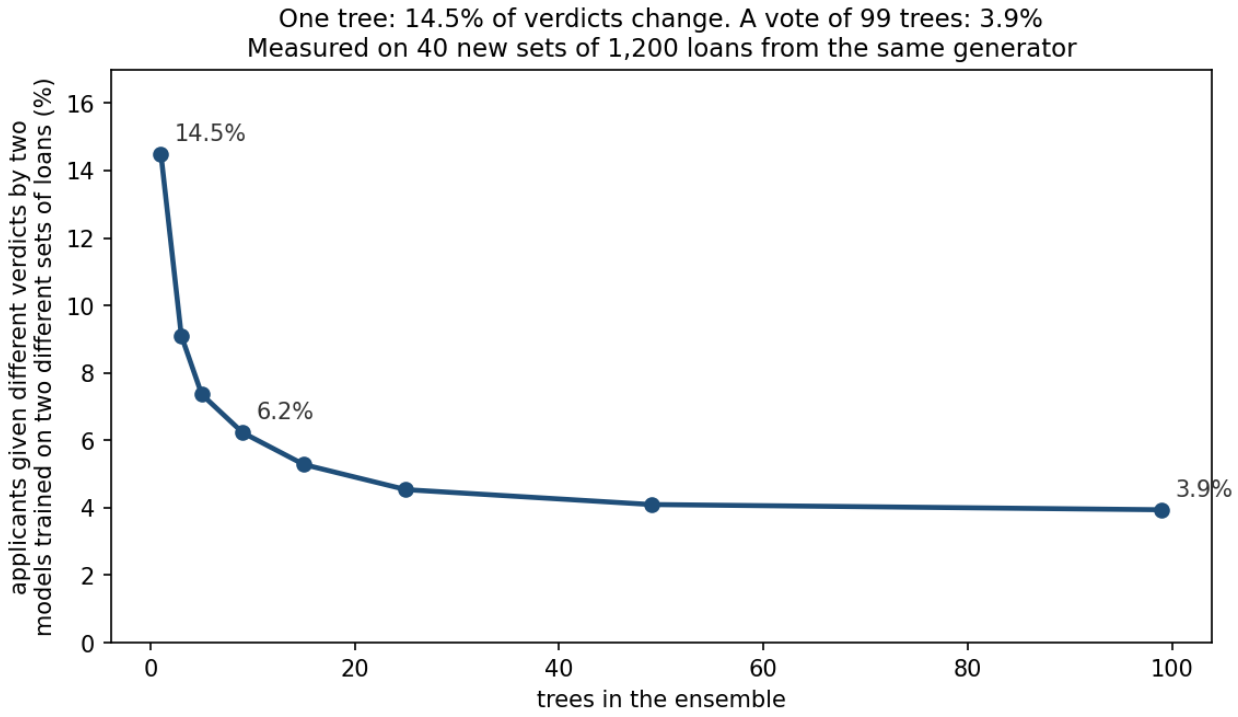
With respect to the other model, not to the truth. Two models, trained on two different sets of loans, are asked about the same applicant; if their answers differ, the verdict changed. It is slide 27's idea, two trees and 140 opposite verdicts, done on many sets and many pairs.

Question — what are the "40 new sets"?

What are the 40 datasets in the figure title?

40 new datasets of 1,200 loans from the same generator, `load_loans` with seeds 1000 to 1039 (the lesson's own uses 42): the same rule that generated the data and the same 7% flip rate, with new applicants and new flipped labels, as if the lender collected 1,200 loans in 40 different years. 40 sets make 780 pairs.

Slide 39 — Another 1,200 loans: how many verdicts change?



Measured on 40 new sets of 1,200 loans: the share of applicants on which two models, trained on two different sets, give different verdicts, for an ensemble of the first 1 to 99 trees.

Read the curve left to right. One tree: the verdict changes on **14.5%** of the applicants, about one in seven. Nine trees: 6.2%. Ninety-nine trees: **3.9%**, and from about 50 trees on the curve is flat. That flat part is the floor of slide 37: what every tree shares because every tree grew from the same 1,200 loans. Past fifty trees, adding more buys almost nothing; the only remaining lever is trees that are less alike.

A predictable mistake. "Where the curve goes flat, the ensemble stops making errors." The curve falls and levels off like a model converging, so the reading is natural. But the vertical axis

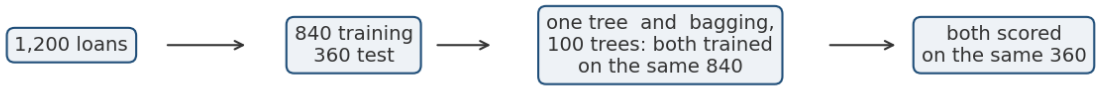
counts verdicts that **change** between two training sets, not **wrong** verdicts: the floor is a floor of instability, and the ensemble is still wrong on about one applicant in ten.

Question — does the flat part mean adding trees never helps?

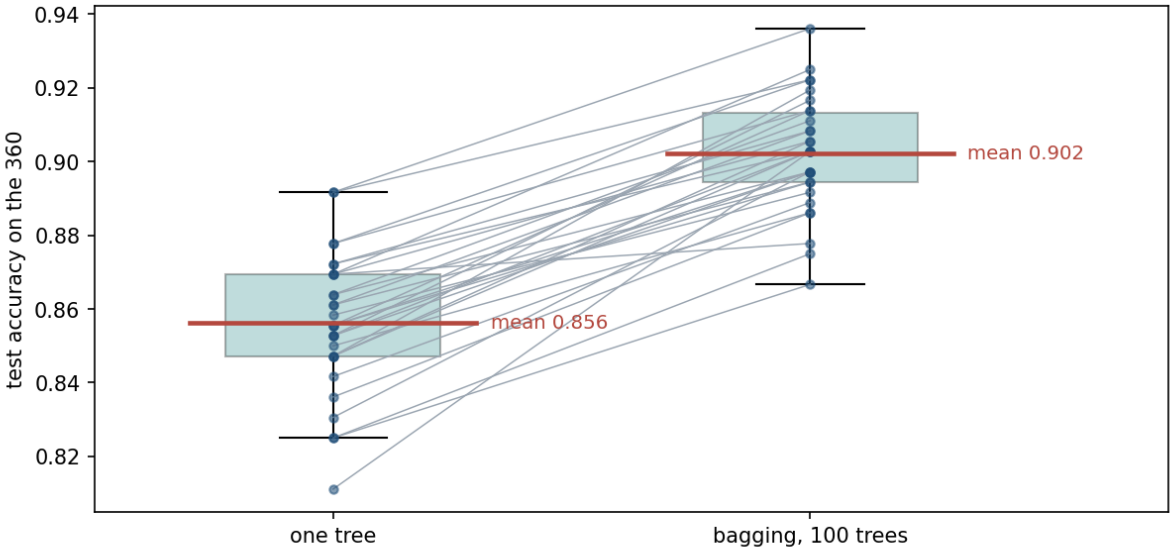
If the curve is flat after 50 trees, should I never use more than 50?

It means that in this experiment, beyond about 50 trees the remaining disagreement hardly changes. It does not fix a number for every dataset; and more trees never hurt a bagging, they only cost time. What the flat part says is that the remaining instability is not the kind more trees can remove.

Slide 40 — Bagging against one tree, on 30 splits



repeated with 30 different random splits; a grey line joins the two models on the same split



box: the middle half of the 30 splits; red line: the mean; dots: the splits

The procedure on top; below, the test accuracy of one tree and of a bagging of 100 trees on 30 random splits. A grey line joins the two models on the same split; the red line is the mean of the 30; the box holds the middle half of the values.

The procedure first, because the result means nothing without it. The 1,200 loans are split at random into 840 for training and 360 for test; one unconstrained tree and a bagging of 100 trees are both trained on the same 840 and scored on the same 360; this is repeated with 30 different random splits.

The message is the mean: 0.856 against 0.902, so on 360 test applicants bagging gets $(0.902 - 0.856) \times 360 = 16.6$, about 17, more right. And split by split, every grey line rises: bagging is ahead on all 30 splits, by 3 to 33 applicants. The single tree's slivers and memorised flipped labels are outvoted. The dots are also a little closer together, standard deviation 0.0153 against 0.0185, but much of that spread is the luck of which 360 applicants land in the test set, which no model removes.

The **box plot**, the first of the lesson: the shaded box holds the middle half of the 30 values, from the 25th to the 75th percentile, and the whiskers reach the most extreme values that are not far outliers.

Question — is the spread about the same?

> *Single tree against bagging, 840 training and 360 test, 30 splits: bagging's accuracy is higher, and the spread is about the same. Right?*

Right. The spread is a little tighter, but the message is the mean, about 17 more right of 360. Because the two models share each split, compare them split by split: every line rises.

Question — why 30 splits and not one?

> *Why repeat the comparison 30 times instead of trusting the first split?*

Because a test accuracy depends on which applicants fall in the test set. One split could favour either model by luck; 30 paired splits show whether the advantage holds across partitions. Here it holds on all 30.

Slide 41 — Random forests: trees less alike

Slide 37 said that to go below the floor the trees must be less alike. A **random forest** is bagging plus one rule: at **every split**, the tree may use only `max_features` columns, drawn at random and **redrawn at every split**. The default is $\sqrt{n}$ rounded down, where n is the number of **columns**; here $n = 2$, so each split is offered **one** column.

The intuition: a committee of experts who all look at the strongest clue first reach the same verdicts and make the same mistakes, so averaging them is like asking one. Force each, at every decision, to choose among a few clues drawn at random, and their reasoning diverges: each is a little worse alone, the committee is better. A crowd is wise only when its members err independently.

On the first of the 30 splits, all 100 bagged trees open on `debt_ratio`; the forest's trees open 51 on `debt_ratio` and 49 on `income_k`. Each forest tree alone is a little worse, 0.828 against 0.837 mean test accuracy, because some splits are denied the better column; the forest as a whole is better.

A predictable mistake. "Every tree is weaker, so the forest must be worse." Right about each tree, wrong about their vote: what a vote needs is trees that err differently, not trees that are each the best possible.

A second one. "n is the number of classes." Reasonable, because here both are 2. But n counts the columns: with the 22 columns of the noise experiment, each split gets $\lfloor\sqrt{22}\rfloor = 4$ columns, while the classes are still 2.

A wording slip. "The forest has more variability." It has more **diversity** between its trees; each tree alone is less steady, and the forest as a whole is steadier.

A note of history. Breiman's paper "Random Forests" appeared in 2001, the same year as his "Statistical Modeling: The Two Cultures", a defence of judging models by how well they predict.

Question — what is the intuition?

Give me the intuition of the random forest.

The committee above. And the mechanism behind it: because the column offered is redrawn at every split, a tree denied `debt_ratio` at the root can still use it further down; it simply builds a different tree, with different mistakes.

Question — why deliberately deny a tree the best feature?

Why make trees worse on purpose?

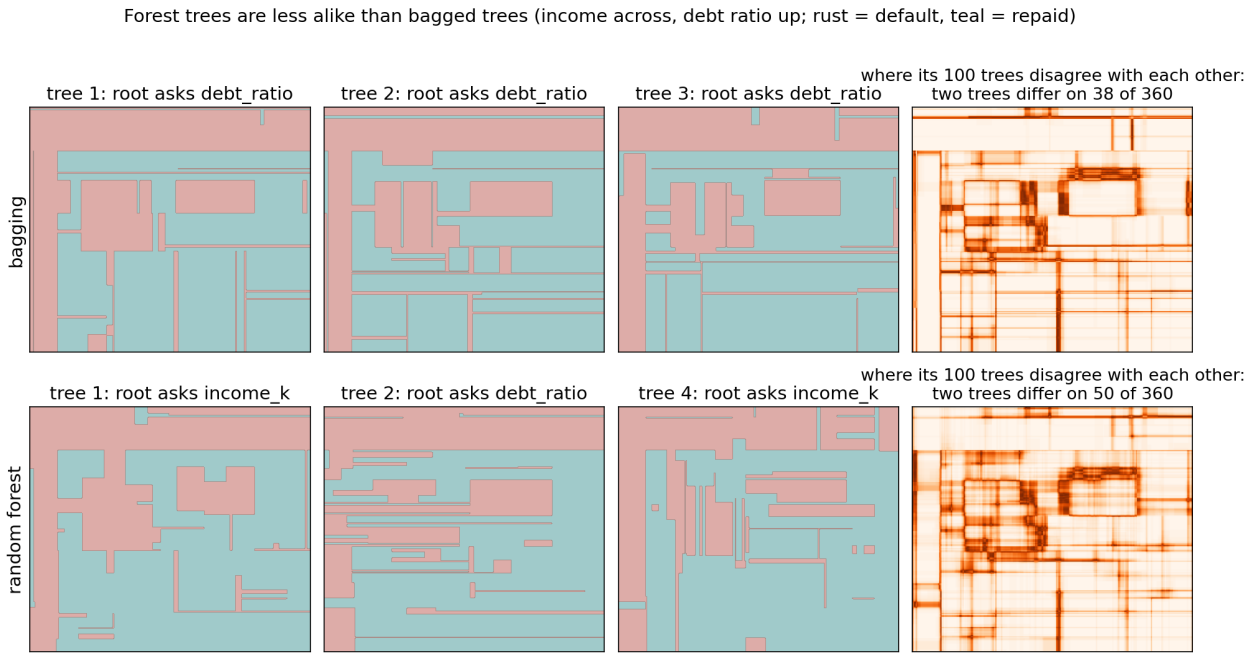
Because the ensemble's floor is what the trees share. Making each tree a little worse, in its own way, shrinks what they share more than it costs each tree. Slide 43 measures the trade: each tree slightly worse, the forest slightly better and steadier.

Question — is more diversity always better?

If less alike is better, should the trees be made as different as possible?

No. Diversity helps only while each tree still carries useful information. Offer each split only random columns when most columns are noise, and many splits have nothing useful to choose from: slide 46 shows the forest's accuracy falling and its importance going to noise.

Slide 42 — Bagged trees against forest trees



Top: three bagged trees and, right, where bagging's 100 trees disagree with each other. Bottom: three forest trees and where the forest's 100 trees disagree with each other.

Both ensembles are trained on the first of the 30 splits. The three bagged trees are its first three: all open on debt and draw the same two strips. The three forest trees are chosen to show both cases, the first that opens on debt and the first two that open on income, and they fill the plane differently: tree 2, offered only debt at many of its splits, is nearly all horizontal bands.

The right column counts disagreement the way slide 27 did for two trees: two bagged trees give opposite verdicts on 38 of the 360 test applicants, on average over all pairs; two forest trees on 50. Less alike, so their mistakes fall in more different places, and the vote has more of them to cancel.

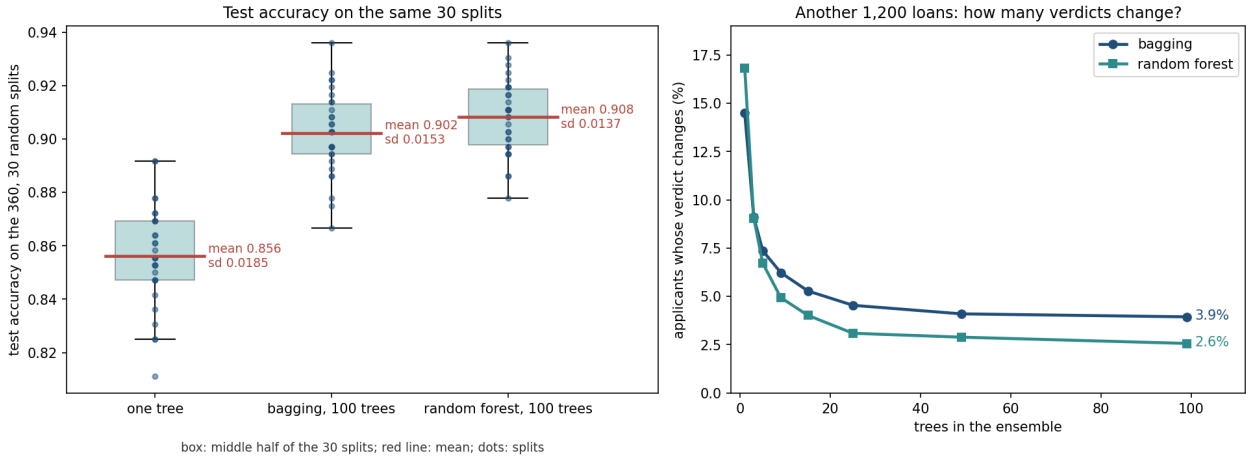
A predictable mistake. Reading the right column as "where bagging and the forest disagree". Each panel looks at one ensemble only: its own 100 trees, disagreeing with each other, darker where the vote is closer to half and half.

Question — so the forest diversifies the errors?

> *So with the random forest the errors are diversified, not all wrong the same way, and so easier to cancel?*

Yes. Each forest tree alone errs a little more, but its errors are less shared, so the vote does better: what counts is how different the errors are, not only how many.

Slide 43 — Random forests, measured



Left: test accuracy on the same 30 splits for one tree, bagging and the random forest. Right: the verdict-change curve of slide 39, with the forest added.

Left. The means are 0.856, 0.902 and 0.908, the standard deviations 0.0185, 0.0153 and 0.0137. The step from one tree to bagging is large, about 17 more of 360 right; the step from bagging to the forest is small, about 2 more. The vote does most of the work; making the trees less alike adds a little more.

Right. With another 1,200 loans, the vote of 99 bagged trees changes its verdict on 3.9% of the applicants, the forest's on **2.6%**: the forest's floor is lower. At the left end one forest tree alone changes more than one bagged tree: each forest tree is less steady, the forest steadier.

Nothing about the splitting rule changed from the first hour. The only change across the three is what surrounds the tree: resampling, then also restricting the columns each split may use.

Question — is the floor now 2.6%, and what are the boxes?

> *Thanks to the forest the floor fell to 2.6%, the part tied to the dataset's peculiarities. The forest has the highest accuracy with a similar spread. What are the green rectangles?*

Essentially yes: 2.6% is what the forest's trees still share, the 1,200 loans' chance features, mostly near the edges of the risky zones. The spread, 0.0137 against 0.0153, is a small difference and partly test-set luck; the right panel is the clearer measure of stability. The rectangles are box plots, as on slide 40: the middle half of the 30 values, with the red line at the mean.

Slide 44 — Feature importance on one tree

The computation behind every importance score, on slide 8's depth-2 tree:

Split	Applicants	Gain ΔG	Weight = applicants ÷ 1,200	Credit = ΔG × weight
debt ratio 0.82 (root)	1,200	0.1438	1.0000	0.1438
income 28 (left)	971	0.0836	0.8092	0.0676
debt ratio 0.947 (right)	229	0.0026	0.1908	0.0005

Debt collects $0.1438 + 0.0005 = 0.1443$, income 0.0676, together 0.2119. Divided by the total so that the columns add up to 1: **debt 0.681, income 0.319**, exactly what `tree.feature_importances_` returns. A forest does this in every tree, averages, and scales again.

Question — why does the root get more credit than the right split?

Why is the root's credit so much larger than the small split on the right?

Twice over. Its gain is larger, 0.1438 against 0.0026; and it splits every applicant, while the right split works on 229 of them, so its gain is weighted by 1 and the other's by 0.1908.

Slide 45 — What a forest's importance says

Feature importance says how much each column helped the trees' questions, scaled so that all the columns add up to 1. Each question makes its groups purer; the improvement is credited to the column the question asks about; added up over all questions of all trees and scaled, it is the column's importance. A 300-tree forest on the 1,200 loans gives **debt 0.61, income 0.39**: with only the two real columns, they share all the credit.

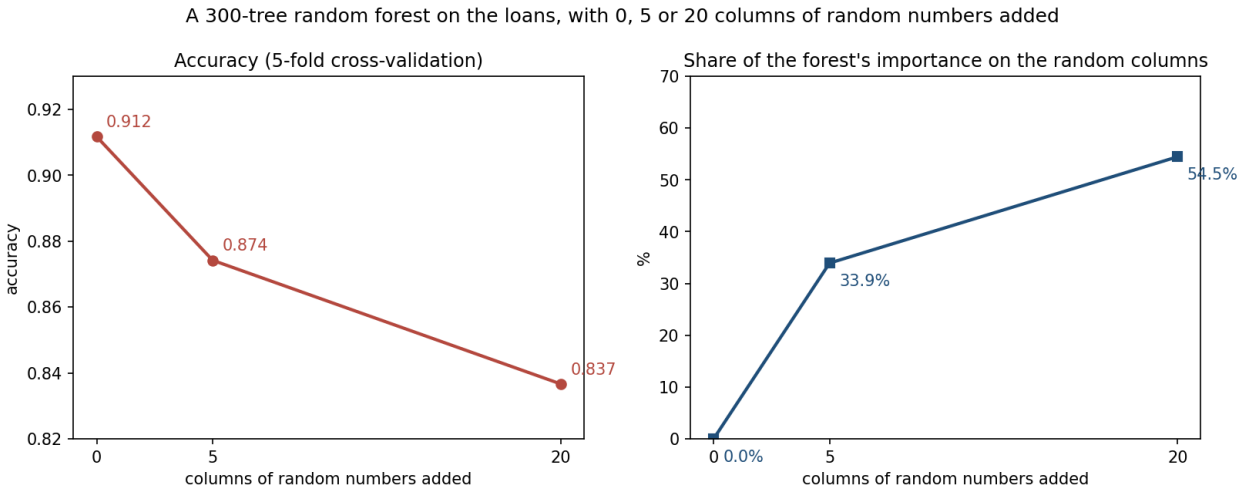
Then the slide asks you to **predict**: add columns of pure random numbers, unrelated to default. How much importance should they get? Most people answer "almost none", because averaging over hundreds of trees should wash out anything with no signal. The next slide is the answer.

Question — is importance the same as correlation with default?

Is a column's importance just how strongly it is correlated with default?

No. It measures how the fitted trees **used** the column: the impurity reductions credited to it at the forest's splits. A column can earn credit by being split on often, even by chance, without being related to default at all, which is what the next slide shows.

Slide 46 — More columns, more noise



A 300-tree forest on the loans with 0, 5 and 20 columns of random numbers added. Left: cross-validated accuracy. Right: the share of the forest's importance that lands on the random columns.

Follow the right panel: 0% with no noise, 33.9% with 5 columns, **54.5%** with 20. With 20 pure-noise columns against 2 real features, more than half of the forest's own importance lands on columns that are random numbers by construction. The left panel falls at the same time, from 0.912 to 0.837, but gently: nothing in the accuracy warns that the importances have gone wrong.

The instinct this breaks is reasonable: averaging over hundreds of trees really does reduce the variance of the **prediction**. It does not guarantee small importance for irrelevant columns. Two things hand credit to noise, the random menu at each split and the deep splits on small groups; slide 65 takes them apart.

Question — what does "more than half of the importance" mean?

What exactly does "over half the importance" mean?

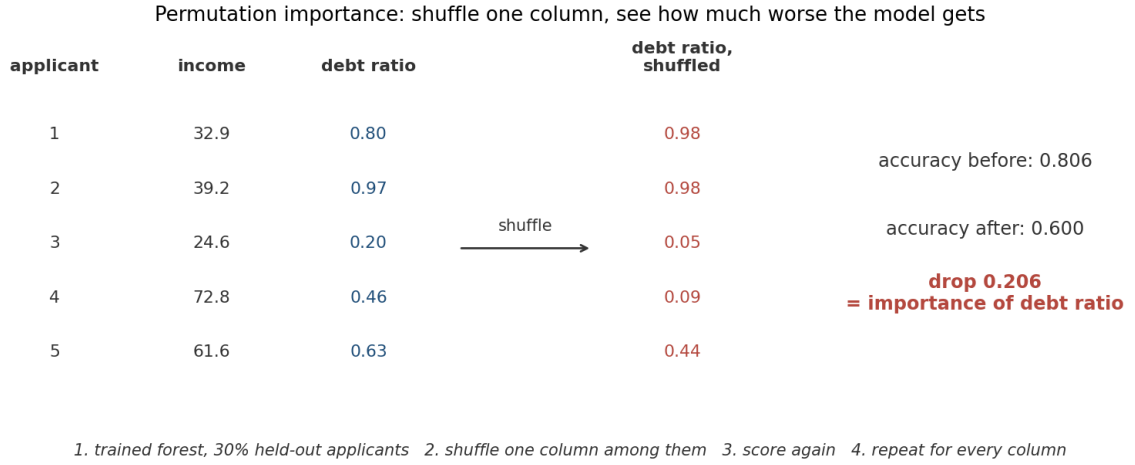
The importances of all 22 columns add up to 1, and the 20 random columns together take 0.545. One by one, the real columns are still the top two: in the forest of slide 48, debt 0.319 and income 0.133, against 0.027 for an average random column. But there are twenty random columns. The problem is the **mass**, not the ranking: a reader who did not know they were noise would conclude they explain more than income and debt together.

Question — why doesn't averaging 300 trees make the noise disappear?

With 300 trees, shouldn't the random columns average out to nothing?

Averaging cancels what differs between trees. The credit given to noise does not differ in sign: every tree that splits on a random column records a positive gain, however small, and positive gains add up. Two causes produce them, the menu that sometimes offers only noise and the small groups deep in each tree; slide 65 has both.

Slide 47 — Permutation importance: the procedure



One column of the held-out applicants shuffled, the forest scored again: the accuracy lost is that column's importance.

The forest's own importance can be wrong; this check measures instead of asking the model. The image: to learn how much an ingredient matters in a recipe, replace it with something random and taste. Four steps:

1. Take the trained forest, with 2 real and 20 random columns, trained on 70% of the loans, and the 30% it never saw.
2. **Shuffle one column** among those applicants, so that each ends up with someone else's value, and leave every other column as it is.
3. **Score the forest again**; the accuracy lost is that column's importance.
4. Put the column back and repeat for every column.

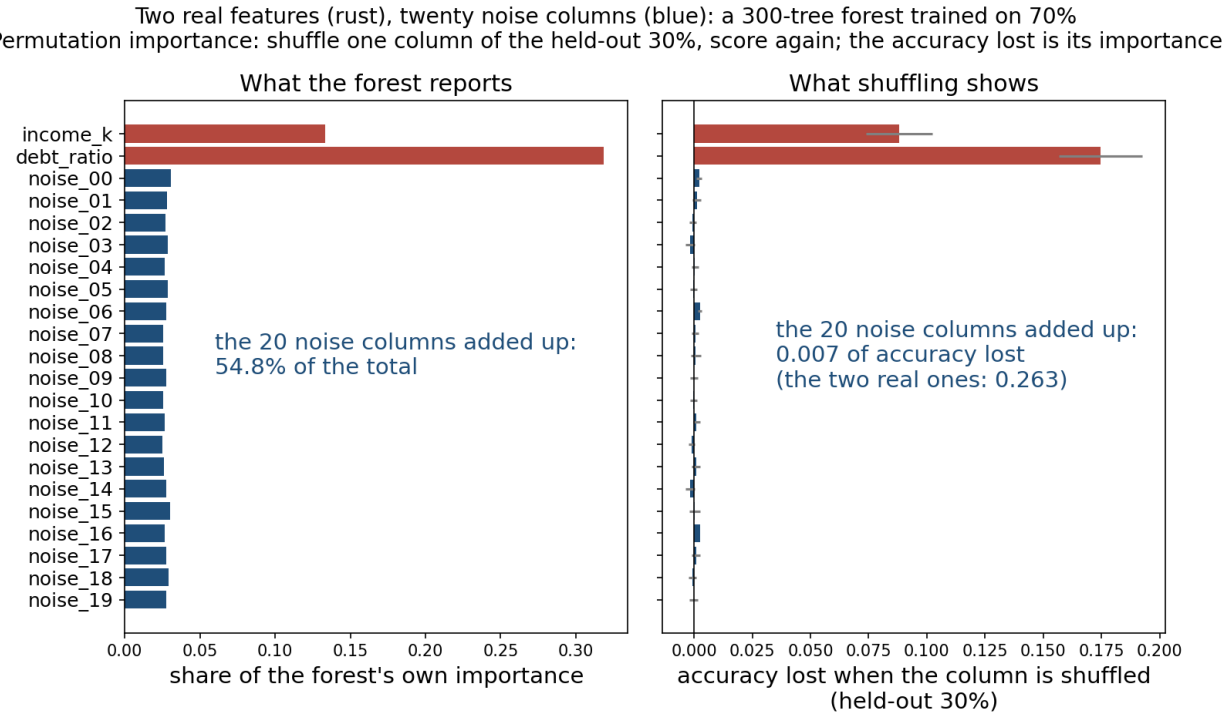
On the slide, one shuffle of the debt ratio takes the accuracy from 0.806 to 0.600, a drop of 0.206. Notebook 2 repeats each shuffle 20 times and averages: 0.175 for the debt ratio.

Question — why shuffle one column at a time?

> *Why not shuffle several columns at once?*

Because the question is about one column: how much the trained model's predictions depend on it. Shuffling it alone breaks its link with the label and with the other columns, while the model and every other column stay exactly as they were, so the drop belongs to that column alone.

Slide 48 — Permutation importance: claimed against measured



Left: the importance the forest reports for itself. Right: the accuracy lost when each column of the held-out 30% is shuffled.

The recipe's result for all 22 columns. **Left**, the forest's own importance: 54.8% of it on the twenty noise columns. (The 54.5% of slide 46 is the same kind of forest trained on all 1,200; two fits, two close numbers.) **Right**, the same columns by permutation: shuffling the debt ratio costs 0.18 of accuracy, income 0.09, and every noise column almost nothing, between 0.003 lost and 0.002 gained.

Two things to notice. **Almost** nothing, not exactly nothing: shuffling is random and the test set is finite, so a useless column moves the score by a few thousandths either way. And **held-out data** is what makes the check honest: on the training rows a deep forest has memorised its noise columns, and shuffling them would cost something there.

The lesson of the two panels: a ranking the model reports about itself is a claim; shuffling on held-out data is the test of it.

A predictable mistake. "Shuffling a column costs nothing, so the forest never used it." Permutation importance looks like a direct measure of use, but it measures something else: how much the **predictions on new data** depend on the column. The forest used the noise columns plenty, as the left panel shows; those splits just do not help on applicants it has not seen.

Question — how can a column have negative importance?

A few noise columns show a small gain when shuffled. Are they useful in reverse?

No. On a finite test set a shuffle can improve the accuracy by a thousandth or two by chance. A small negative value is what "no importance" looks like once noise is added to it.

Slide 49 — Notebook 2, live

The notebook's map: bagging from scratch against `BaggingClassifier` (0.872 both, and 1 of 29 flipped labels right); the out-of-bag score against cross-validation (0.9083 and 0.9025); the three models over 30 splits (0.856, 0.902, 0.908); the roots of the bagged and the forest trees (100 against 51 and 49); the noise-column experiment (54.5% on noise); the permutation check (real columns 0.09 and 0.18, noise about 0).

The cell worth protecting is the noise-column sweep: add the columns yourself and watch the share climb. If time allows, try `max_features=None` in section 6 and predict the noise share first; the answer is on slide 65. The atoms notebook has the out-of-bag vote by hand, a `BaggingClassifier` on twelve loans, and importance and permutation by hand.

Part III — Trees in sequence: boosting

Slide 50 — Boosting: one tree after another

The third way of combining trees, and the opposite of the second. Bagging and random forests grow their trees **independently**, then let them vote. **Gradient boosting** grows them **in sequence**: each new tree is trained to correct what the trees before it still get wrong.

Each boosted tree is **shallow**, depth 2 or 3 here, and outputs a **number**, a correction, not a class. So it is a **regression tree**: the same greedy search over features and thresholds, but each leaf predicts the average correction of the examples in it, and a split is scored by how much it lowers the squared error around those averages. Even `GradientBoostingClassifier` grows regression trees inside.

The **learning rate** α decides how much of each correction is added. It is the same symbol as lesson 3's learning rate, doing an analogous job: how large a step each correction takes.

A predictable mistake. "Each tree corrects the previous tree." Natural, because "sequence" suggests a chain of pairs. Each new tree corrects what the **whole sum so far** still gets wrong: start from F_0, the mean; $F_1 = F_0 + \alpha h_1$; tree 2 is trained on what F_1 still gets wrong; the final prediction is the start plus α times all the corrections.

A second one. "The task is classification, so each boosted tree votes default or repaid, like a tree in a forest." Every ensemble so far was made of trees that vote. But a boosted tree never outputs a class: it outputs a correction added to a running score, and only the final score is turned into a class.

A note of history. The first boosting algorithm to work in practice was AdaBoost, by Yoav Freund and Robert Schapire (1995–1997), which reweighted the examples the trees so far got wrong; it won them the Gödel Prize in 2003. Jerome Friedman's 2001 paper recast the idea as gradient descent on a loss, the version this lesson teaches and scikit-learn implements.

Question — is that the idea?

> *Trees are grown in sequence; the next one is trained on the errors of the previous one; each tree outputs a correction, a number, so it is a regression tree; and there is a learning rate here too. Right?*

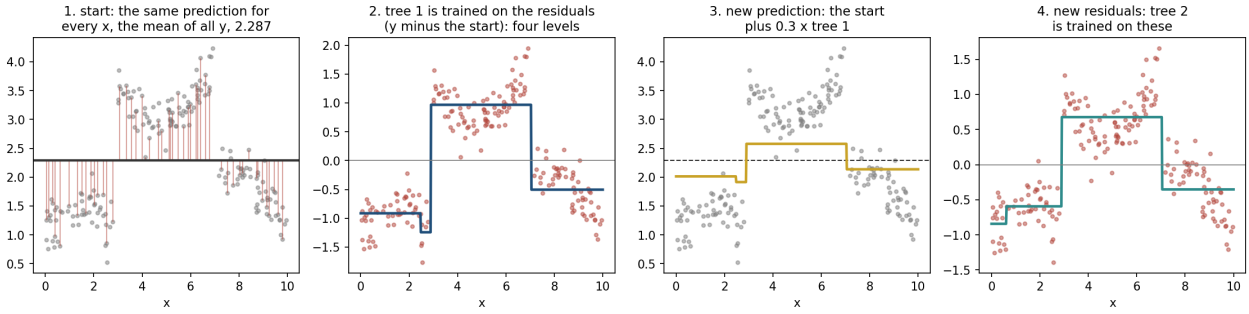
Right, except "the errors of the previous one": of the whole sum so far, as above.

Question — why regression trees inside a classifier?

> *If the task is to classify, why are the trees regression trees?*

Because what each tree predicts is a correction to a score, a number like +0.613 or −0.387, not a class. The score is turned into a probability, and the probability into a class, only at the end (slide 56).

Slide 51 — Boosting, drawn: the first two steps



The procedure on a toy dataset of 200 noisy points: the start, the first tree fitted to the residuals, the new prediction, and the new residuals the second tree is trained on.

Before any formula, the procedure, on 200 noisy points around a step plus a sine wave.

1. **The start.** The same prediction for every x: the mean of all 200 y, 2.287. Before any tree the model cannot tell one x from another, and among all constants the mean has the smallest squared error. The red segments are the residuals, truth minus prediction: negative where the points lie below the line, positive on the high step.
2. **Tree 1**, depth 2 with four leaves, is trained on those residuals, not on y. It splits x at 2.48, 2.89 and 7.05 and predicts −0.914, −1.243, +0.967 and −0.504.
3. **The new prediction** is the start plus 0.3 times tree 1: at $x = 5$, $2.287 + 0.3 \times 0.967 = 2.578$. It moves only part of the way, because the learning rate is 0.3.
4. **The new residuals**, and tree 2 is trained on these.

The average size of a residual falls from 0.838 to 0.601 after tree 1. And so on, sixty times.

Question — what does "a flat prediction, the mean of all y" mean?

> *What does "the model starts from a flat prediction: the mean of all y" mean?*

Before any tree, the model has no way to tell one x from another, so it gives everyone the same number: a horizontal line. The best such number for squared error is the mean, 2.287, as with lesson 3's model with no features. The residuals around it carry the pattern tree 1 learns: negative on the low step, positive on the high one.

Question — why doesn't the first tree move the prediction all the way?

> *Tree 1 knows the residuals. Why does the prediction move only part of the way to them?*

Because the tree's output is multiplied by the learning rate, 0.3. The correction is deliberately partial, so that later trees can refine what is left; a full step would also copy the noise in the residuals in one go.

Worked example — three boosting steps on four points

Four points: $x = 1, 2, 3, 4$ with $y = 1, 1, 3, 3$. Learning rate $\alpha = 0.5$; each tree has one split, between $x = 2$ and $x = 3$, and predicts the mean residual on each side. Start: the mean, $F_0 = 2$ for every point.

tree	residuals left, right	tree's output left, right	new prediction left, right
1	-1.000, 1.000	-1.000, 1.000	1.500, 2.500
2	-0.500, 0.500	-0.500, 0.500	1.250, 2.750
3	-0.250, 0.250	-0.250, 0.250	1.125, 2.875

Each tree is trained on what the sum so far still gets wrong, and each step covers half of the remaining distance, because $\alpha = 0.5$: the residual goes 1, 0.5, 0.25. The prediction approaches 1 and 3 without ever jumping there in one step.

Slide 52 — Building the ensemble, one tree at a time, for squared error

The previous figure in symbols:

$$F_0(x) = \bar{y}, \qquad F_t(x) = F_{t-1}(x) + \alpha\, h_t(x)$$

F_0 is the flat start, the best single constant for the loss being used: for squared error, the mean of the target; for classification it will be the log-odds of the base rate (slide 56). Each F_t is the previous ensemble plus a scaled correction from the newest tree, h_t. Start flat, then keep adding small, scaled corrections.

Question — only formulas?

> *This slide has only formulas; the procedure on the trees is not understandable from it.*

That is why slide 51 now comes first: the four panels are this formula, drawn. F_0 is the dashed line at 2.287; h_1 is the four-step function of tree 1; F_1 is the dashed line moved by 0.3 times it.

Slide 53 — What each tree is fit to

- For squared error, the next tree h_t is trained on the **residual** $y - F_{t-1}(x)$: what the trees so far have not explained.
- In general, h_t is trained on the **negative gradient** of the loss with respect to the prediction: the direction in which each prediction should move.
- With $L = \frac{1}{2}(y - F)^2$, $-\partial L/\partial F = y - F$: the same thing as the residual.

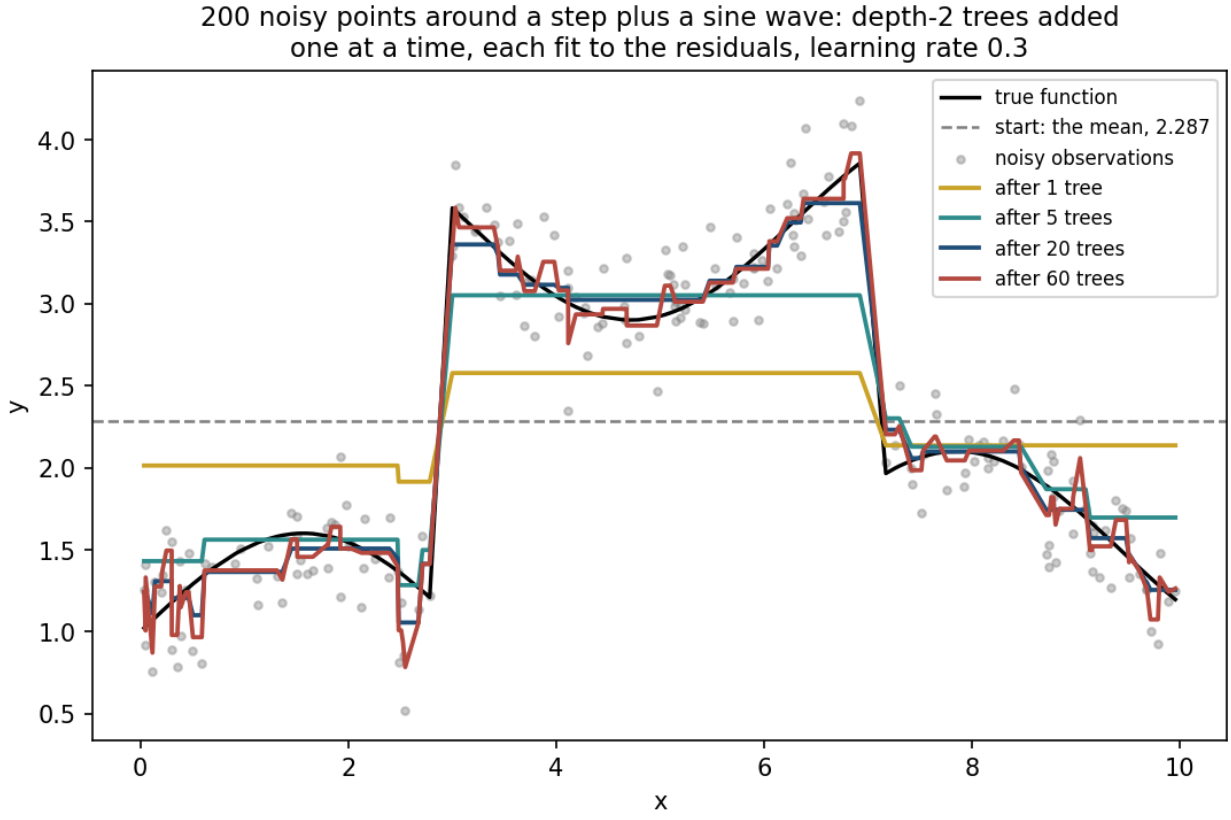
The second bullet is lesson 3's gradient descent, moving the **predictions** of the points instead of coefficients, with a tree to draw that move as a function of x. It matters because it covers classification too, once the loss changes (slide 56). In code, notebook 3's whole loop is a `DecisionTreeRegressor(max_depth=2)` fitted to `y - prediction`, 0.3 times its output added to the prediction, sixty times.

Question — why is the residual the negative gradient?

Why does the derivative give back the residual?

Differentiate $L = \frac{1}{2}(y-F)^2$ with respect to F: the result is $F-y$. Its negative is $y-F$, the residual. The one half is there only to cancel the 2 that the derivative brings down.

Slide 54 — A step plus a sine wave, fit by 60 shallow trees



200 noisy points (grey) around a three-level step plus a sine wave (black); depth-2 trees added one at a time, learning rate 0.3. The dashed line is the start; the curves are the prediction after 1, 5, 20 and 60 trees.

The dashed grey line is the start, the mean 2.287. One tree barely moves it: the gold curve is the start plus 0.3 times tree 1, $2.287 - 0.274 = 2.01$ on the left, $2.287 + 0.290 = 2.58$ in the middle, $2.287 - 0.151 = 2.14$ on the right. Five trees sketch the coarse shape. Twenty trace the step closely. By sixty the ensemble is tracking individual noisy points as well as the shape.

Question — is the mean visible here?

> *Is the starting mean visible on this slide?*

As the dashed line, yes. The gold curve "after 1 tree" is already corrected, but it is the mean shifted by 0.3 times tree 1, which is why it sits at 2.01, 2.58 and 2.14.

Question — how can shallow trees draw a curve?

> *Each tree is a few steps. How can sixty of them follow a sine wave?*

Each tree adds a piecewise-constant correction, a few steps. Summed, many small steps can approximate a curve as closely as you like, the way a staircase with small enough steps looks like a ramp. That is also why the fitted curves are step-shaped up close.

Slide 55 — Training error keeps falling; the true error does not

Trees	Mean squared error (MSE) on the 200 training points	MSE against the true function
1	0.476	0.395
5	0.119	0.068
20	0.046	**0.012**
60	**0.025**	0.018

A number on each curve of the previous slide, two ways. The middle column is the error on the points the model is trained on; the right column is the error against the black line, the true function, which is possible only because these data are synthetic. The training error keeps falling; the error against the truth falls until about 20 trees and then rises: by sixty, the extra trees are chasing noise. Counted tree by tree, the lowest error against the truth is at 19 trees, 0.0116 against 0.0119 at 20, so "about 20" is the honest reading of the table.

The noise added to the points has variance $0.25^2 = 0.0625$, and from 20 trees on the training error is below it. That is a **sign** that the model has started to learn the noise, not a proof: 200 noise values have a sample variance of their own. The proof is the right column, which turns up.

It is the overfitting story of the unconstrained tree, reached by a different route: not one tree grown too deep, but too many shallow trees each chasing whatever residual noise is left. With real data the truth is unknown, and cross-validation takes its place (slides 58 and 59).

Question — how can the true error rise while the training error falls?

> *If the model fits the training points better and better, how can it get worse against the truth?*

Because it is fitted to the noisy points, not to the black line. Once the shape is captured, what is left in the residuals is mostly noise; later trees reduce the training error by following that noise, and every step towards a noisy point is a step away from the line.

Slide 56 — Classification: the same idea, a different gradient

For classification, y is 0 or 1. The running score F becomes a probability through lesson 4's logistic function, $p = \sigma(F) = 1/(1 + e^{-F})$. The loss is lesson 4's **log-loss**, and its negative gradient with respect to F is $y - p$, the **pseudo-residual**: each tree is trained on it.

The first step in numbers. 464 of the 1,200 defaulted, so the ensemble starts at the base rate's log-odds, $F_0 = \log(464/736) = -0.461$, which the logistic function turns back into $p = 0.387$ for everyone. The first tree is fitted to $y - p$: **+0.613** for every applicant who defaulted, **−0.387** for every one who repaid.

What is genuinely different from lesson 4's logistic regression, given that both descend against the same quantity? Lesson 4 moves a fixed number of coefficients; boosting adds new trees, so the model's structure itself grows with training.

A predictable mistake. "So the update is $p + \alpha(y - p)$." Reasonable, because $y - p$ is a difference of probabilities. But the trees correct the **score** F, not p: F moves by α times the tree's output, and only then does the logistic function turn the new F into a new p. That is what keeps p between 0 and 1 however large the corrections add up.

A second one. "The pseudo-residual is a property of the applicant, like income." It is printed per class at the start, +0.613 and −0.387, so it looks fixed. It is recomputed from the current ensemble at every round: as p moves, $y - p$ moves, and each tree is trained on the corrections still needed.

A detail, for anyone reading scikit-learn's source: the splits are chosen on $y - p$, but the number written in each leaf is then recomputed with one Newton step on the log-loss. Nothing in the lesson depends on it; handout section 9 has it in a sentence.

Question — is this sigma the one of the variance formula?

> *Handout section 5.2 uses sigma squared for a variance. Is it related?*

No. Here σ is lesson 4's logistic function. The σ^2 of section 5.2 is the variance of one tree's verdict, a different quantity that happens to share the letter.

Worked example — where classification boosting starts

Ten loans, 3 defaulted. The start is the same for every loan:

- $p = 3/10 = 0.3$, the share who defaulted;
- the score $F_0 = \log\left(p/(1-p)\right) = \log(0.3/0.7) = -0.847$, and the logistic function turns it back into $1/(1 + e^{0.847}) = 0.3$;
- the first tree is trained on $y - p$: $1 - 0.3 = +0.7$ for each defaulter and $0 - 0.3 = -0.3$ for each loan repaid.

The tree's output moves the score F, not p; the next pseudo-residuals are computed from the new p, so they change from round to round.

Slide 57 — On the loan data

`GradientBoostingClassifier` at scikit-learn's defaults, **100 trees** of depth 3 and learning rate $\alpha = 0.1$, on all 1,200 loans and the same five groups of 240 as slide 36: **0.902 ± 0.020** cross-

validated accuracy. It is ahead of the depth-8 tree (0.882) without being told anything about depth, and level with the nine-leaf tree (0.901): on this rectangle data a well-budgeted tree is hard to beat on the score alone.

Question — which data do the results refer to?

Which data are these results computed on?

All 1,200 loans, in five-fold stratified cross-validation with the same five groups of 240 used for every model of the lesson. Each round trains the default boosting on 960 loans and scores the other 240; 0.902 is the mean of the five scores and 0.020 their standard deviation.

Question — why is the default already competitive?

Why does boosting do well with no tuning at all?

Because its defaults are a sensible combination: shallow trees, which cannot memorise much each, and a modest learning rate, which keeps every correction small. On these data 100 such corrections capture most of the strips and the islands.

Slide 58 — Boosting never stops fitting on its own

`n_estimators` ($\alpha = 0.3$, depth 3)	Training accuracy	Cross-validated
5	0.840	0.846 ± 0.025
10	0.888	0.870 ± 0.024
20	0.930	0.895 ± 0.027
30	0.948	**0.897 ± 0.020**
80	0.978	0.893 ± 0.012
200	1.000	0.895 ± 0.017
800	1.000	0.890 ± 0.019

Unlike bagging, boosting keeps reducing its training loss for as long as it runs, because every new tree is built to reduce what remains, including the 7% of labels that are noise. Nothing in the algorithm says "stop". Training accuracy reaches 1.000 by 200 trees; cross-validated accuracy peaks at 30 trees, 0.897, and drifts down to 0.890 at 800: under a point. The unconstrained single tree lost three points past its own peak; boosting's overfitting is real but gentler.

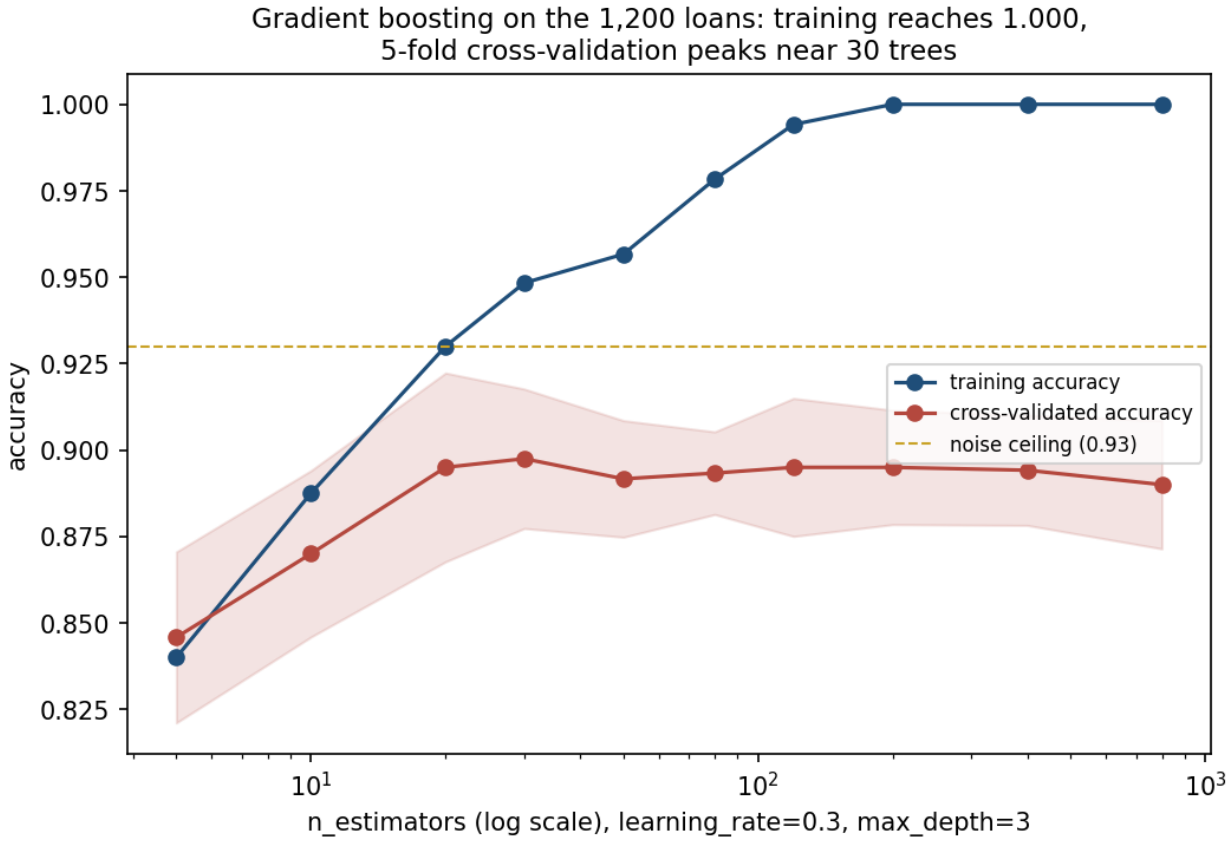
A predictable mistake. "30 trees beat the default 100 of the previous slide." Reasonable, since 30 is the peak here. But this table uses a learning rate of 0.3 and the default uses 0.1: two different configurations, not one knob turned.

Question — why can training reach 1.000 while cross-validation falls?

How can the training accuracy be perfect while the cross-validated one goes down?

Because the extra trees spend themselves fitting the flipped labels of the training folds. Those flips are random, so what is learned about them does not carry over to the held-out fold: the training score rises and the held-out score drifts down.

Slide 59 — Training climbs to 1.000 and stays; the honest curve turns over first



Training accuracy and five-fold cross-validated accuracy against the number of trees (log scale), learning rate 0.3, depth 3; the noise ceiling dashed.

The same story as the depth curve of slide 23, reached by adding trees instead of depth, and the same as the step and sine wave, with cross-validation in place of the true function. A training accuracy of 1.000 is far above the noise ceiling, so the flipped labels have been learned by heart. Cross-validated accuracy climbs quickly, peaks near 30 trees, and drifts down over the following 770.

The damage per added tree is small and easy to miss in one run, which is why the number of trees must be chosen on held-out data, never left to run.

Question — how do I choose when to stop, if the truth is unknown?

> *On real data there is no black line. How do I know when to stop adding trees?*

Two ways. This lesson cross-validated a list of tree counts and kept the best, 30. The other is **early stopping**: set aside part of the training data, watch the score on it as trees are added, and stop once it has not improved for a number of rounds. In scikit-learn that is the `n_iter_no_change` argument of `GradientBoostingClassifier`.

Slide 60 — Fixing 120 trees, varying only the learning rate

Learning rate α	Training accuracy	Cross-validated
0.02	0.888	0.872 ± 0.025
0.05	0.914	0.897 ± 0.022
0.10	0.939	**0.902 ± 0.022**
0.30	0.994	0.895 ± 0.020
1.00	1.000	0.878 ± 0.020

Learning rate and tree count both control how much total correction the ensemble applies, so they trade off. At $\alpha = 0.02$, 120 trees have not finished fitting the signal: training accuracy is still under 0.89. At $\alpha = 1.00$, the ensemble has memorised the training set and given back more than two points of cross-validated accuracy, 0.902 down to 0.878. The best sits in between: large enough to make progress within the tree budget, small enough that any one correction is easy for later trees to undo if it turns out to be wrong. The more cautious, "more regularised" direction is the **small** α: the same inverted-direction trap as lesson 6's C.

A predictable mistake. "Only α times the number of trees matters, so half the learning rate with twice the trees gives the same model." Natural, since both knobs set the total correction. But each tree is fitted to what the ensemble before it still gets wrong, so many small steps follow a different path from a few large ones. A large step overshoots on the noisy labels and the next trees spend themselves undoing it; a small step lets later trees correct an early mistake. The trade-off is real; the equivalence is not.

Slide 61 — Notebook 3, live

The notebook's map: boosting from scratch on the residuals (error against the truth 0.0119 at 20 trees); classification with `GradientBoostingClassifier` (start −0.461, then 0.902); the tree-count sweep (peak 0.897 at 30); the learning-rate sweep (best 0.902 at 0.1); and the leaderboard of the next slide. The cell to protect is the residual sweep: watching the error against the truth fall and rise again is the most direct experience of boosting's overfitting. The atoms notebook does the first two boosting steps by hand and checks them against `GradientBoostingRegressor`.

Part IV — Closing

Slide 62 — Every method, on the same five folds

Model, on the 1,200 loans	Cross-validated accuracy	± over the 5 folds
Majority baseline	0.613	
Logistic regression	0.748	0.030
Single tree, unconstrained	0.852	0.021
Single tree, `max_depth = 8` (chosen on these folds)	0.882	0.010

Model, on the 1,200 loans	Cross-validated accuracy	± over the 5 folds
Gradient boosting, 30 trees (chosen on these folds)	0.897	0.020
Single tree, nine leaves	0.901	0.020
Bagging, 100 trees	0.904	0.020
Random forest, 100 trees	**0.911**	0.018
noise ceiling	*0.93*	

Everything from the lesson, cross-validated the same way, on the same folds. The ± is the standard deviation of the five fold scores, the spread lesson 5 asks to see beside every mean. The ensembles land within about a point and a half of each other, the random forest on top, and one single tree sits among them: the nine-leaf tree, at 0.901. On this dataset that is no accident: the rule is made of axis-aligned rectangles, the shape a few well-placed thresholds draw.

An honest caveat, in lesson 5's terms. The depth of 8 and the 30 boosted trees were picked because they scored best on these same five folds, so those two rows are slightly optimistic; the table says so. Bagging and the random forest ran at their defaults and chose nothing; the nine leaves were chosen for readability, not on the folds.

A predictable mistake. "Gradient boosting is the best." Reasonable: it is presented last, and on tabular data in practice it often wins. Here the random forest is on top and boosting at 30 trees is fourth: the most sophisticated method is not automatically the best.

Question — is the forest definitely better than bagging?

> *0.911 against 0.904: can I conclude the forest is better?*

On these folds it has the higher mean, but the gap is small against the spread of about 0.02: the top four rows lie within one spread of each other. The table ranks the methods in this experiment; it does not prove a general superiority.

Slide 63 — Three findings

- **On this dataset** the three ensembles land within 1.5 points of each other, and a **nine-leaf tree** among them (0.901): one a person can read.
- What the ensembles buy here is **steadiness**: over 30 splits the nine-leaf tree's spread is 0.020, the forest's 0.0137 (means 0.897 and 0.908).
- The **unconstrained tree is the worst tree**, though the most flexible. (It is still far above logistic regression and the baseline.)

The third finding repeats a pattern seen with several methods: flexibility without a stopping rule is not an advantage. The degree-12 polynomial of lesson 3, $k = 1$ in lesson 6, and today both the unconstrained tree and the boosted ensemble that kept adding trees past its peak.

Question — why is a nine-leaf tree so close to the ensembles?

> *If ensembles are the point of the lesson, why does one small tree almost match them?*

Because the rule that generated the data is a few axis-aligned rectangles, which a small tree can draw almost exactly. On such data the ensembles' advantage is less a higher score than a steadier one: their verdicts move less when the data change (slide 43).

Slide 64 — How to choose, in practice

Situation	Reach for
The decision must be explained to the person it affects	A single tree, small enough to read; if even that is too opaque, none of these models
Tabular data, accuracy matters most, no explanation required	A random forest: the best score here, at its default settings
Accuracy matters, and there is time to tune the number of trees and the learning rate together	Gradient boosting, tuned on validation data
Many columns, few of them informative	Any tree method, but check its importances by permutation
Too few rows to set a validation set aside	A random forest, judged by its out-of-bag score

Each row rests on evidence from today: the nine-leaf tree read as if/else rules (slide 25); the leaderboard (slide 62); the tree-count curve and the learning-rate table (slides 59 and 60); the noise experiment and the permutation check (slides 46 to 48); the out-of-bag slides (34 and 35). On the first row: explaining a credit decision to the person it affects is also a legal question, the subject of the lesson's optional reading, `Resources/right_to_explanation.md`.

Question — is the random forest always the best choice for tabular data?

Should I just always use a random forest on tables?

No. It is the strongest method in this lesson's comparison, at its defaults, which makes it a good first try. The table shows situations where a readable tree or a tuned boosting is the better choice; the choice depends on what the task requires as well as on the score.

Slide 65 — 54% of the importance on noise

The lesson's number: with **20 random columns against 2 real ones, more than half of a random forest's importance landed on the random ones.** Every lesson so far has left one of these numbers: lesson 1's 77% accuracy on coin-flip labels, lesson 3's 365. This is lesson 7's.

Two causes, not one.

- **The menu.** Each split may use $\lfloor\sqrt{22}\rfloor = 4$ of the 22 columns. The chance that all 4 are noise is $\binom{20}{4}/\binom{22}{4} = 4{,}845/7{,}315 = 0.662$: two splits in three see no real column. At those splits the tree still has to pick something, and it picks the noise column that happens to separate the classes best.
- **Small groups.** With every split offered all 22 columns (`max_features=None`), the noise share falls from 54.5% only to **28%**. Deep in a tree a node holds a handful of applicants, and

among twenty columns of random numbers one will almost always separate that handful a little better than income or debt can, purely by chance.

The same run raised the accuracy from 0.837 to 0.897: with 20 useless columns out of 22, the random menu costs more accuracy than decorrelating the trees buys back. The default is a good default, not a law. The check is the one of slide 48: shuffling a column on held-out data costs the two real features 0.09 and 0.18 of accuracy, and each noise column almost nothing.

A predictable mistake. "So 54% of the forest's predictions are decided by noise." Reasonable, since a share of importance reads like a share of responsibility. But the 54% adds up credit for splits, the impurity gain each bought on the training rows; it is not a share of predictions or of accuracy. Permutation importance measures that second thing, and there the noise columns are worth almost nothing.

Question — why does noise still get 28% with every column on offer?

If every split can choose among all 22 columns, a real one is always available. Why does noise still take 28%?

Near the root a real column always wins. Deep in the trees, a node holds a few applicants, and the real columns have little left to say about them; among twenty random columns, one will separate those few slightly better by chance. Each such gain is small, but there are thousands of them across 300 trees, and they add up.

Worked example — how often the menu holds no real column

A smaller version of slide 65: 2 real columns and 6 random ones, 8 in all; each split is offered 2 of them at random.

- Possible menus: $\binom{8}{2} = 28$.
- Menus with no real column, both picked among the 6 random ones: $\binom{6}{2} = 15$.
- Share: $15 \div 28 =$ **0.536**: more than half of the splits must pick a random column.

The lesson's case is the same count with 20 random columns out of 22 and 4 offered: 4,845 ÷ 7,315 = 0.662.

Slide 66 — What to take away

The lesson's story in six sentences:

1. A tree asks **one question at a time**, the one that best separates default from repaid.
2. **Depth** is the dial: too shallow misses the rule, too deep learns the noise.
3. **One tree is unstable**: another sample, other verdicts.
4. Many trees on different samples **vote** away each tree's bad luck; a random forest makes them **less alike**, so the vote is steadier.
5. **54%** of a forest's importance landed on 20 noise columns: check importance by **shuffling** columns on unseen data.
6. **Boosting** chains trees, each correcting what is left; **stop it** on unseen data.

If one habit is to stay: before trusting a random forest's importance ranking, ask how many of the columns could plausibly matter at all, and check the ranking by shuffling on held-out data.

Next lesson turns from labelled to unlabelled data: every model so far learned from examples whose answer was known; lesson 8 asks what structure there is in data when nobody has given the answer.

Slide 67 — Homework

Exercise 7, `Exercises/07_trees_and_ensembles.md`, is discussed at the start of lesson 8. It gives you two loan products from the same lender, on which the ranking of the methods is not the same, and asks you to predict which method wins before fitting anything, to measure it, and to explain the difference with at least one measurement per product, not an assertion. As always for the exam, the reasoning in your notebook counts more than the score: report every number with its cross-validated spread, and treat any feature-importance ranking as a claim to check, not a fact to quote.